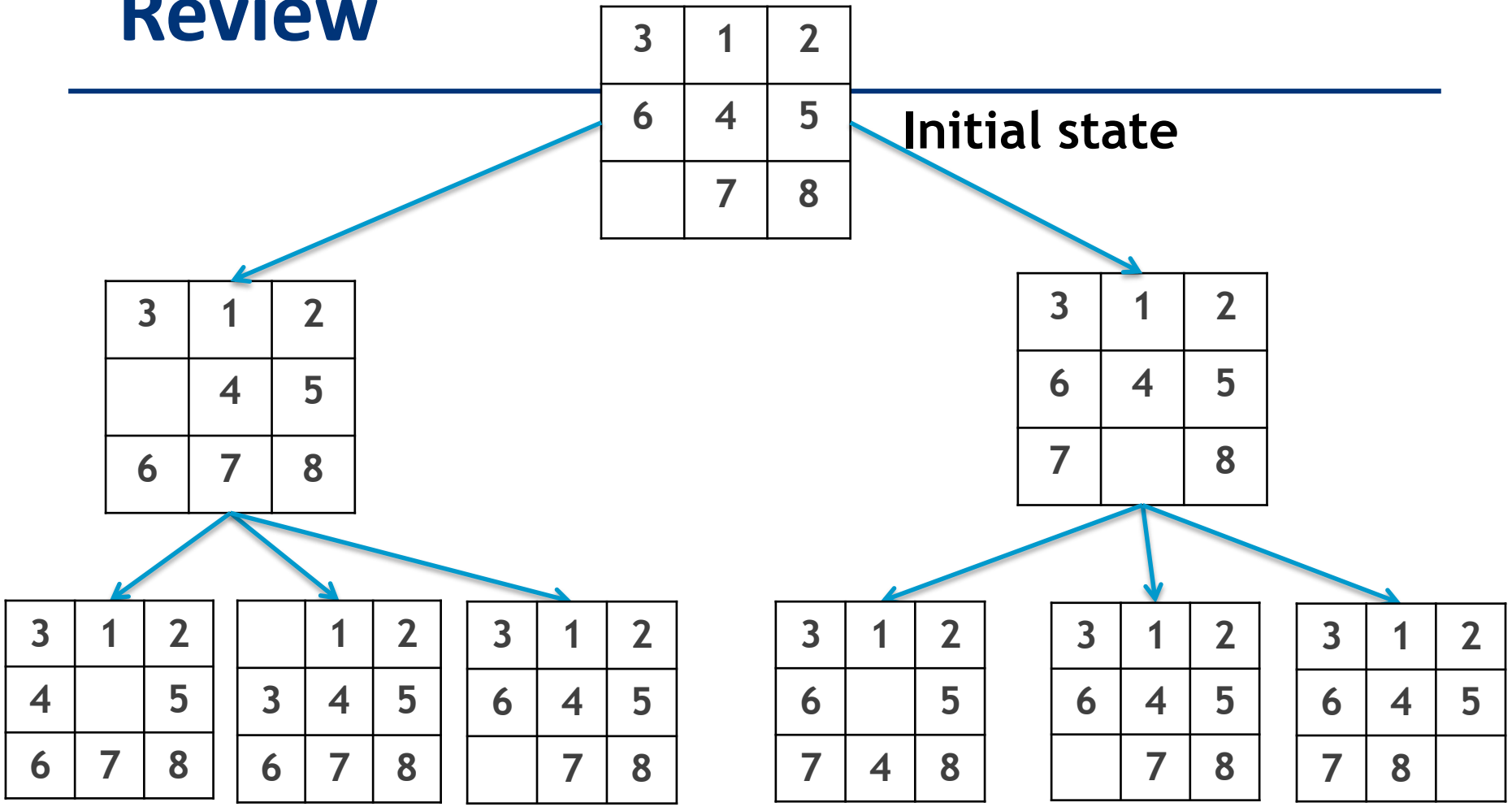
CS 380: ARTIFICIAL INTELLIGENCE

Lecture 4: Informed Search, A*, & Heuristics

Where we are…

- Problem Solving:
  - Problem Formulation
- Search: finding a solution
  - Uninformed search
  - Informed search

Review



Set of states reachable by executing two actions

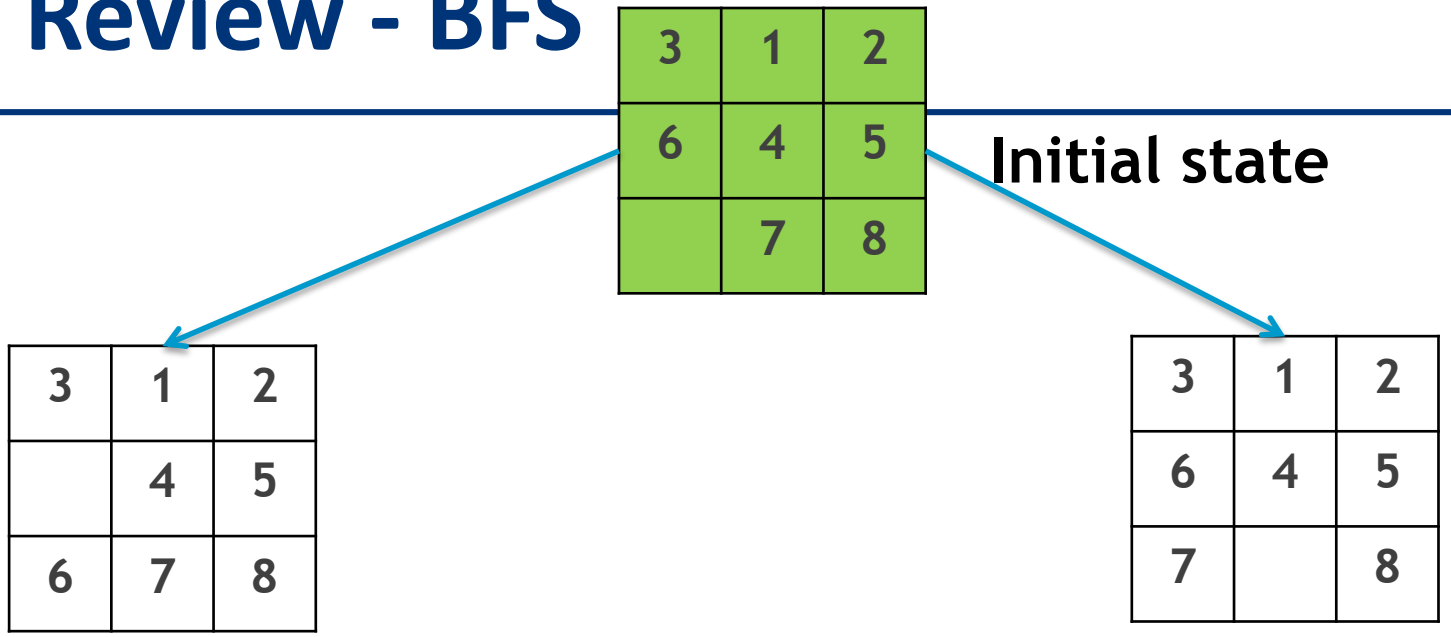
Review - BFS

3	1	2
6	4	5
	7	8

Initial state

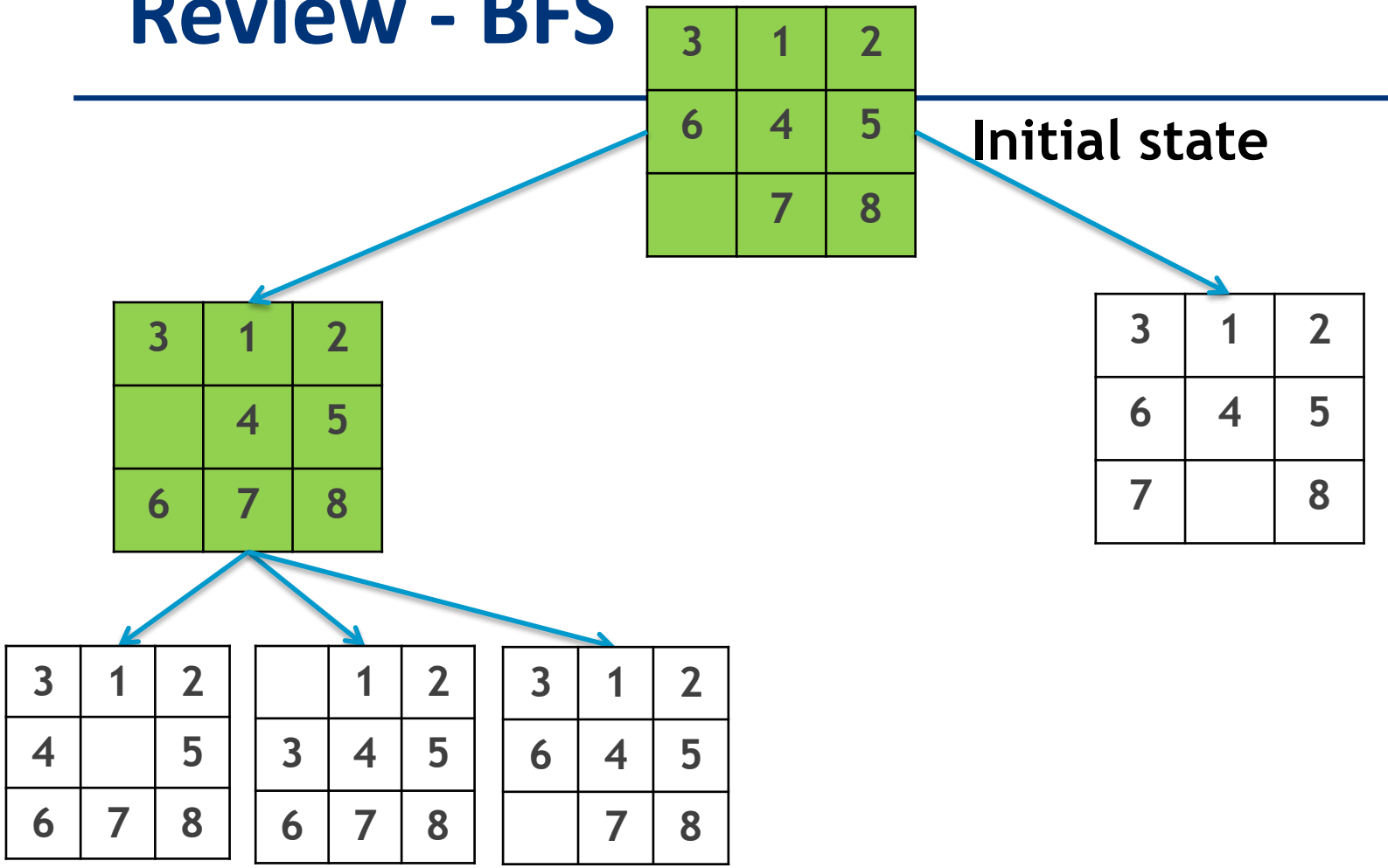
Set of states reachable by executing two actions

Review - BFS



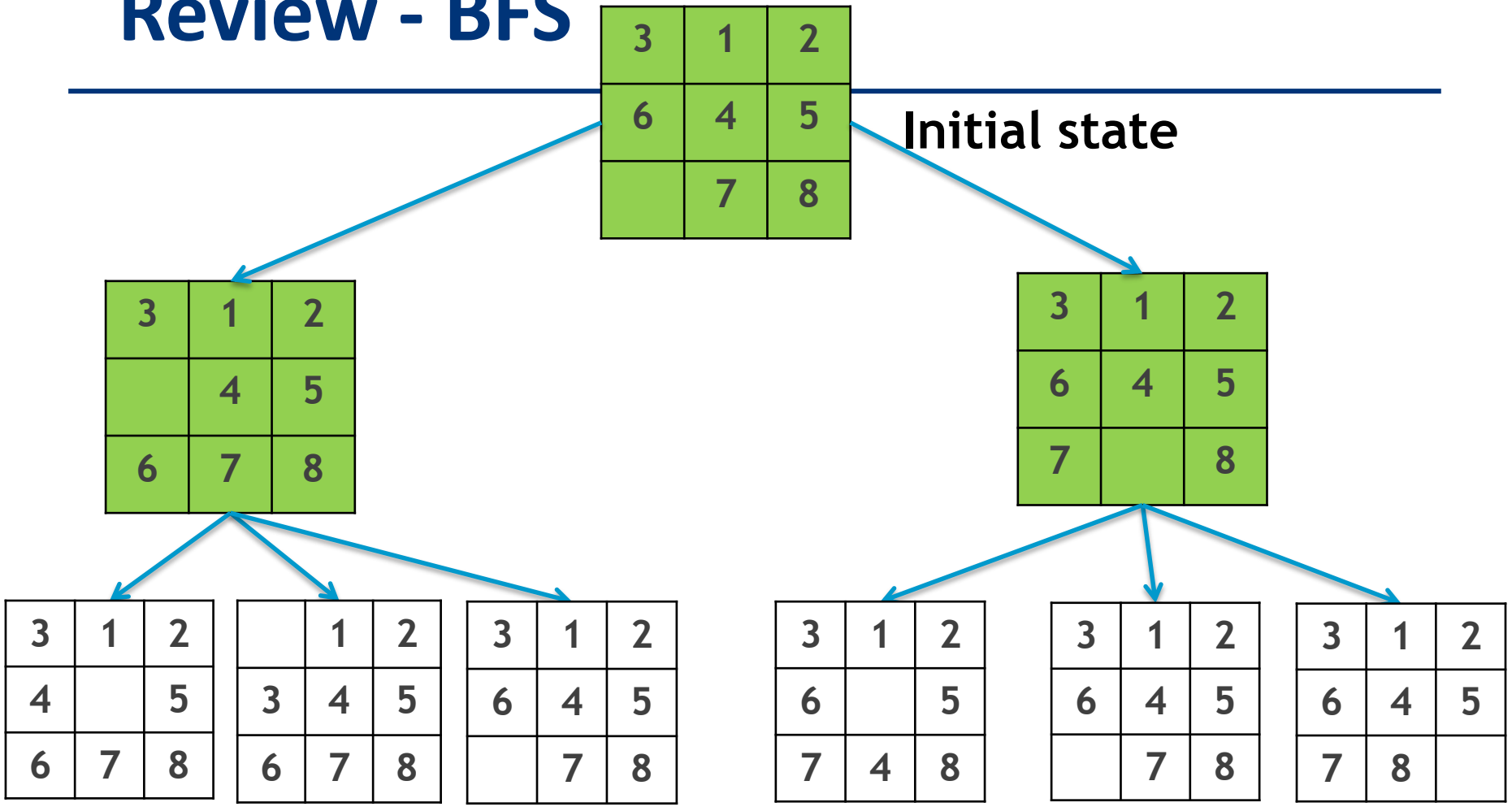
Set of states reachable by executing two actions

Review - BFS



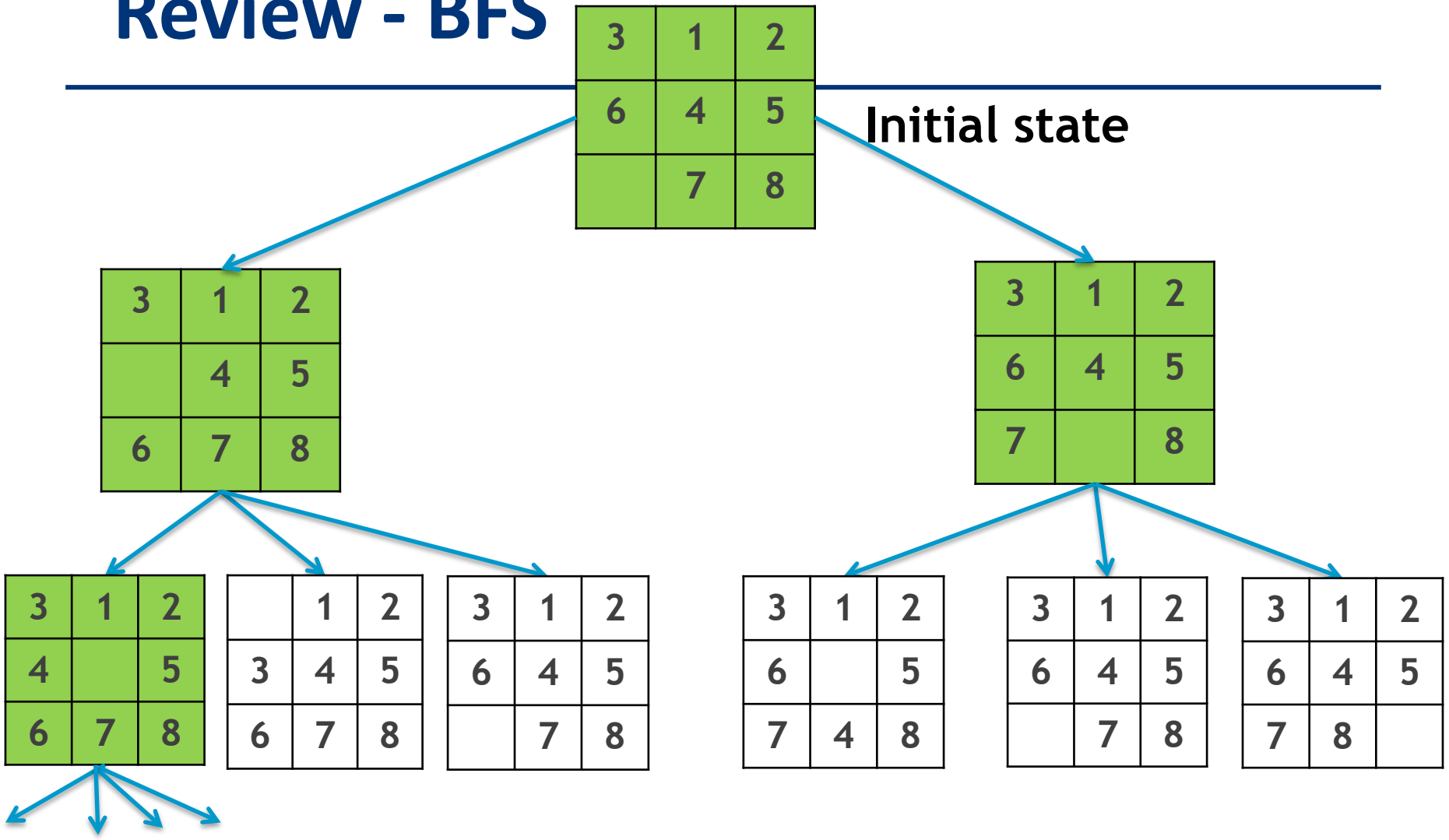
Set of states reachable by executing two actions

Review - BFS



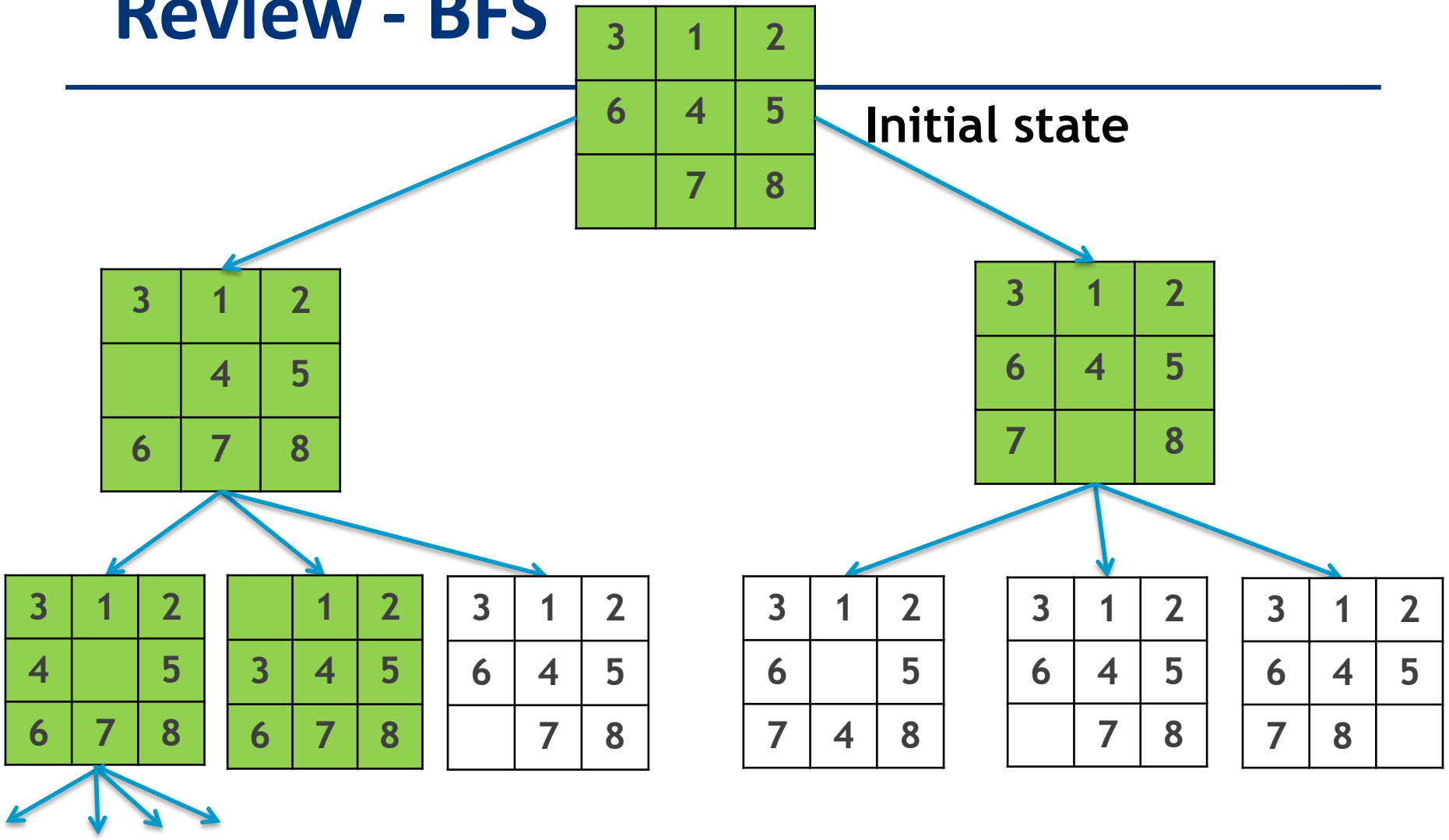
Set of states reachable by executing two actions

Review - BFS



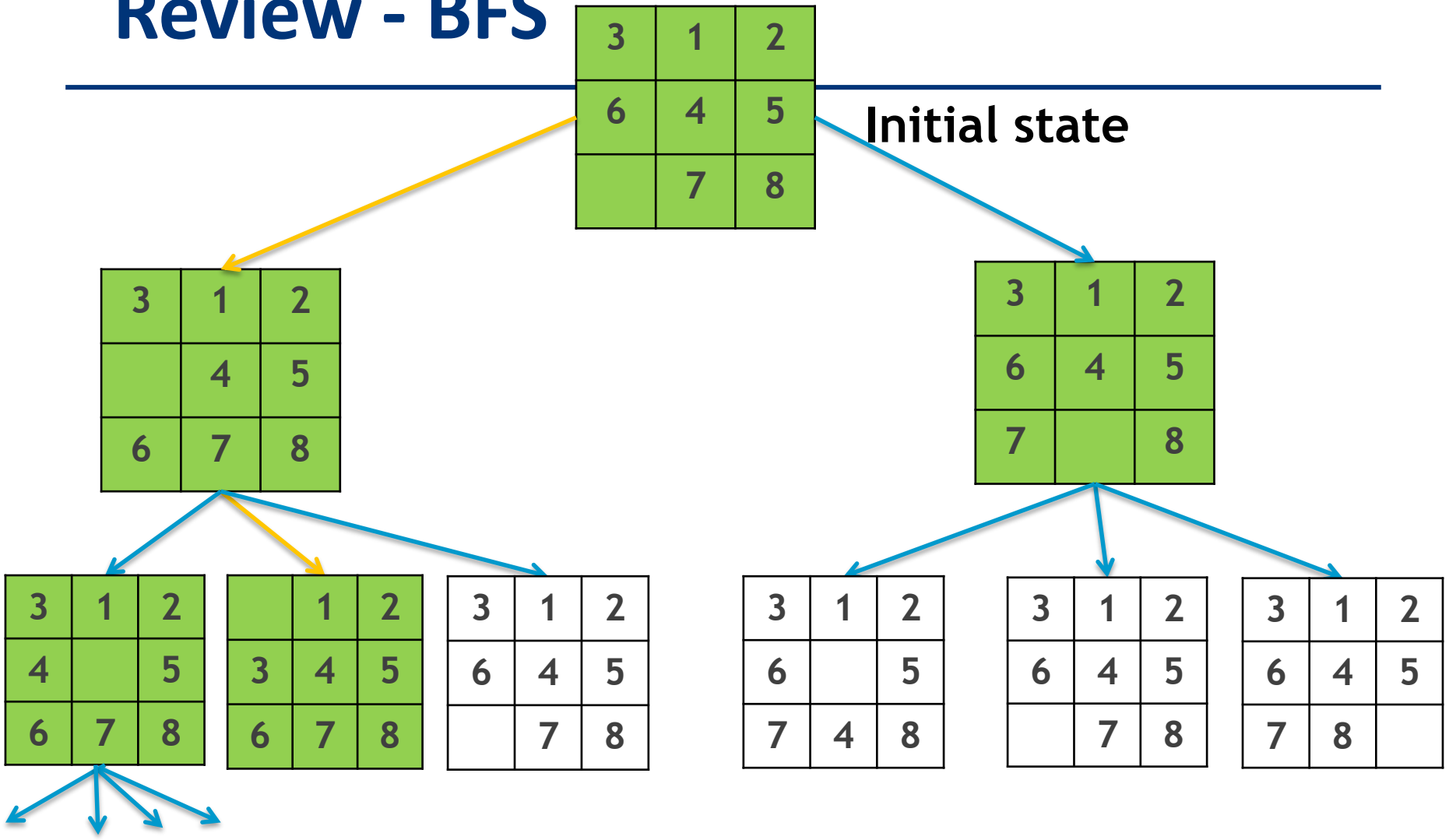
Set of states reachable by executing two actions

Review - BFS



Set of states reachable by executing two actions

Review - BFS



Set of states reachable by executing two actions

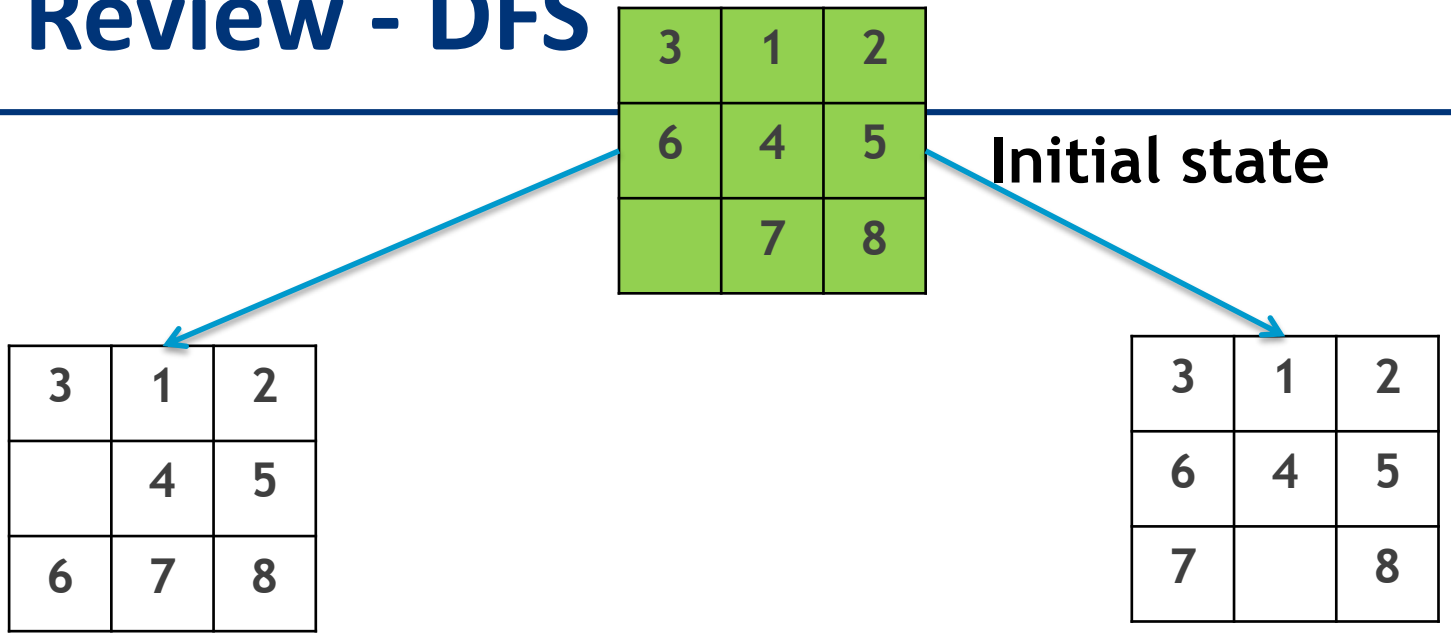
Review – DFS

3	1	2
6	4	5
	7	8

Initial state

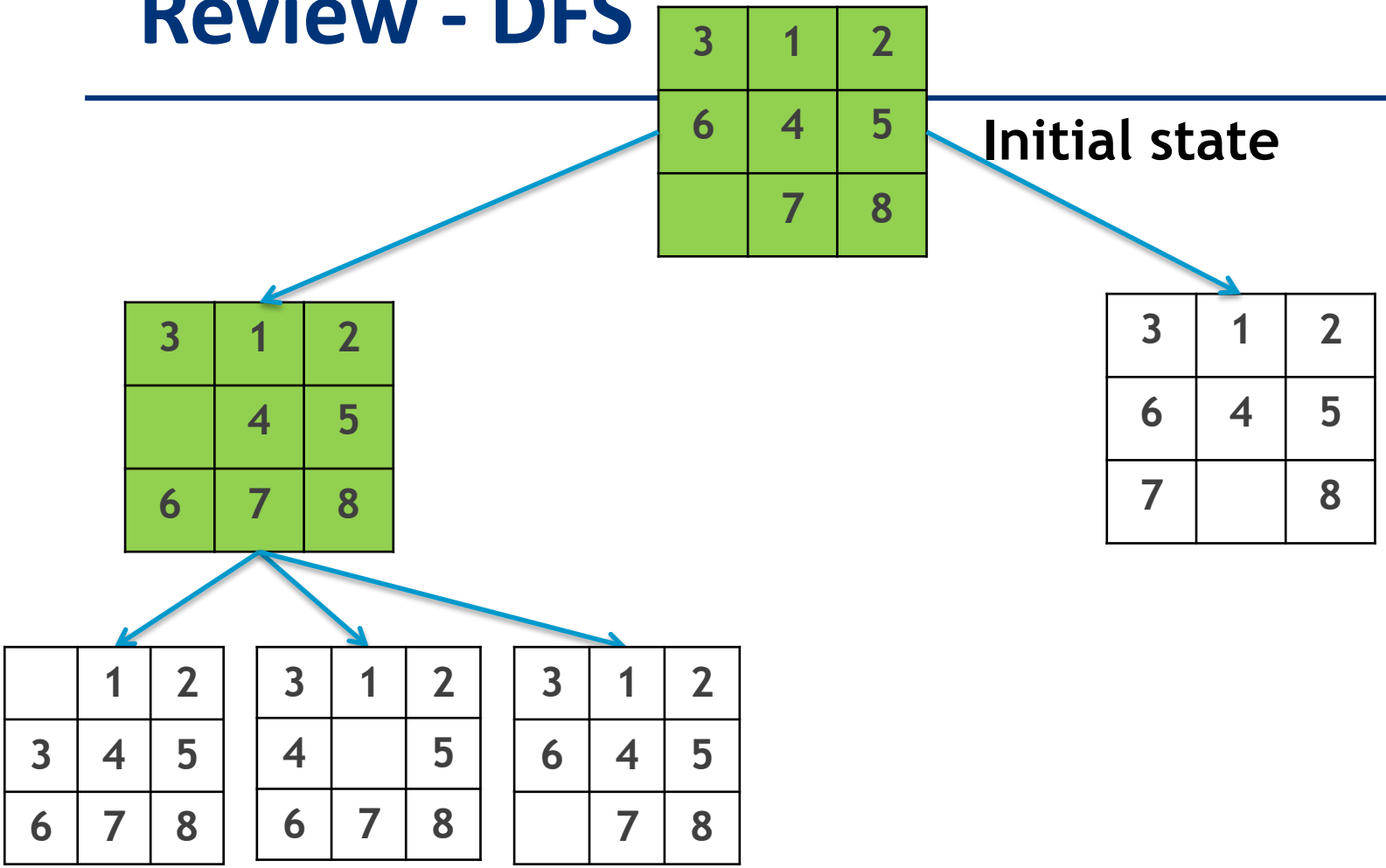
Set of states reachable by executing two actions

Review - DFS



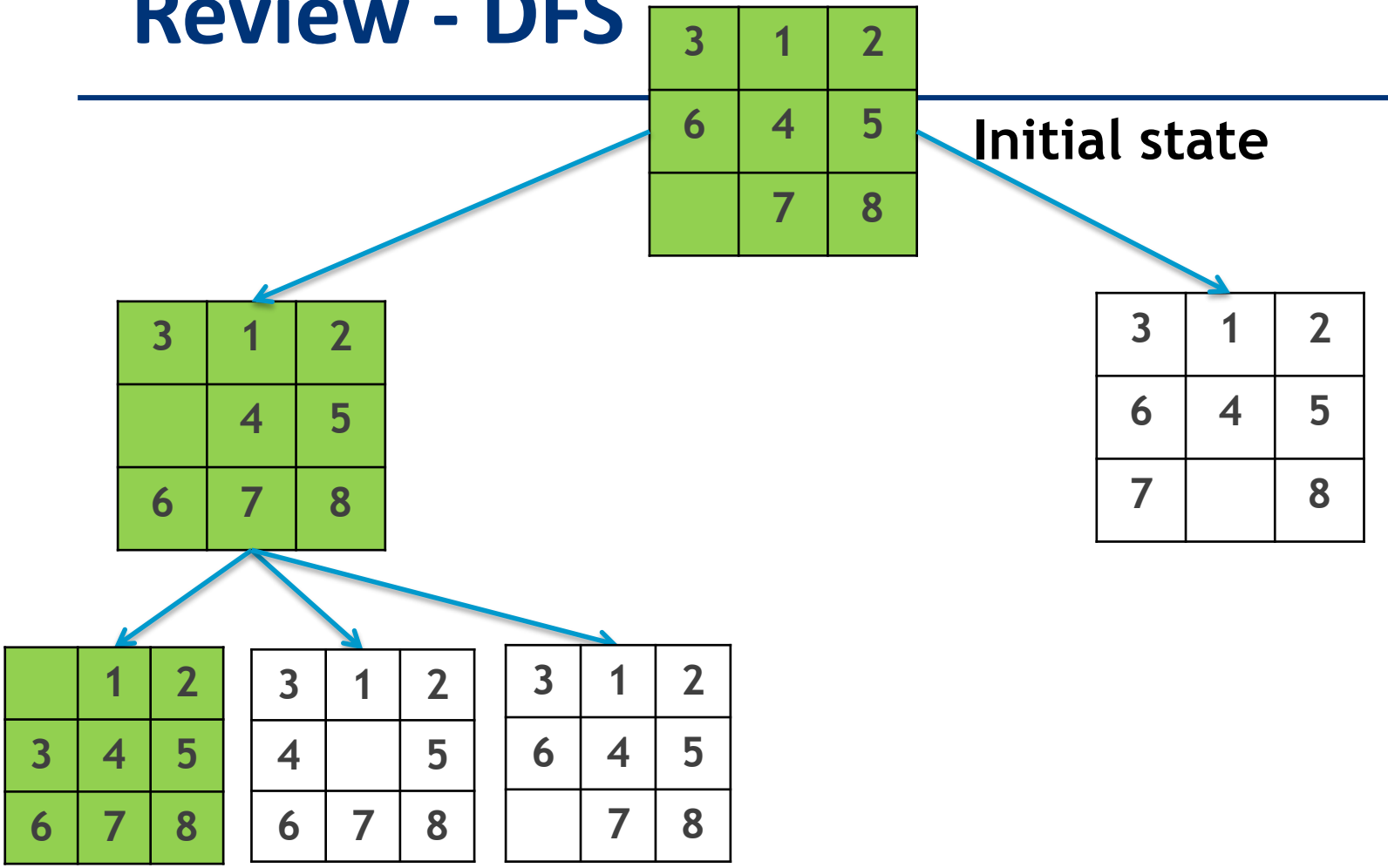
Set of states reachable by executing two actions

Review - DFS



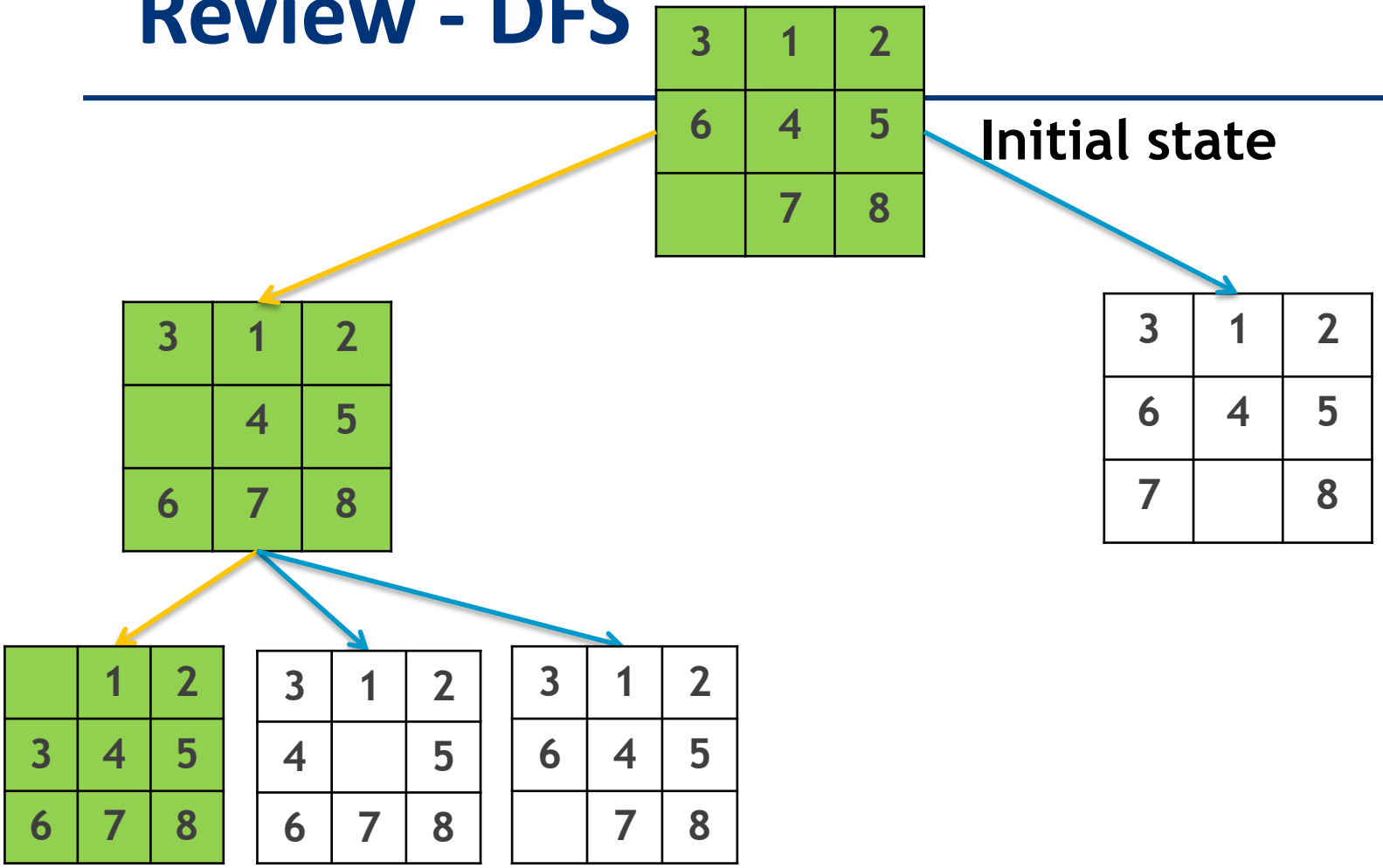
Set of states reachable by executing two actions

Review - DFS



Set of states reachable by executing two actions

Review - DFS



Set of states reachable by executing two actions

Review

- **OPEN** (or Fringe) list = nodes queued for processing
- **CLOSED** list = nodes/states we've already processed

- Breadth-first Search
  - Remove the first node in the list
  - Check if that first node's state is a goal state
    - If it's a goal, you're done
    - If not, add children to ***end*** of OPEN list and repeat
- Depth-first Search
  - Same as BFS, except add children to ***start*** of OPEN list.

- How can you check if a state is a repeated state?
  - Check whether it's already in either OPEN or CLOSED

Graph Search

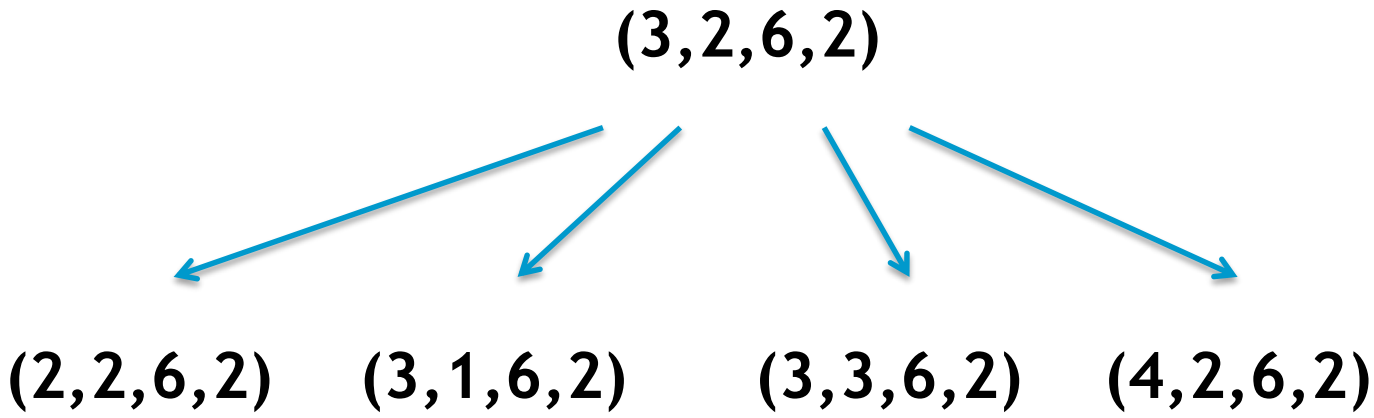
```
function Graph-Search(problem, fringe) returns a solution, or failure
    closed ← an empty set
    fringe ← Insert(Make-Node(Initial-State[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← Remove-Front(fringe)
        if Goal-Test(problem, State[node]) then return node
        if State[node] is not in closed then
            add State[node] to closed
            fringe ← InsertAll(Expand(node, problem), fringe)
    end
```

Uninformed → Informed Search

- All the problem-solving strategies we have seen so far use the following knowledge:
  - **Goal check**: knowing if the problem is solved
  - **States**
  - **Actions**: an agent knows which actions can be executed in the current state.

- Note that this is very little information

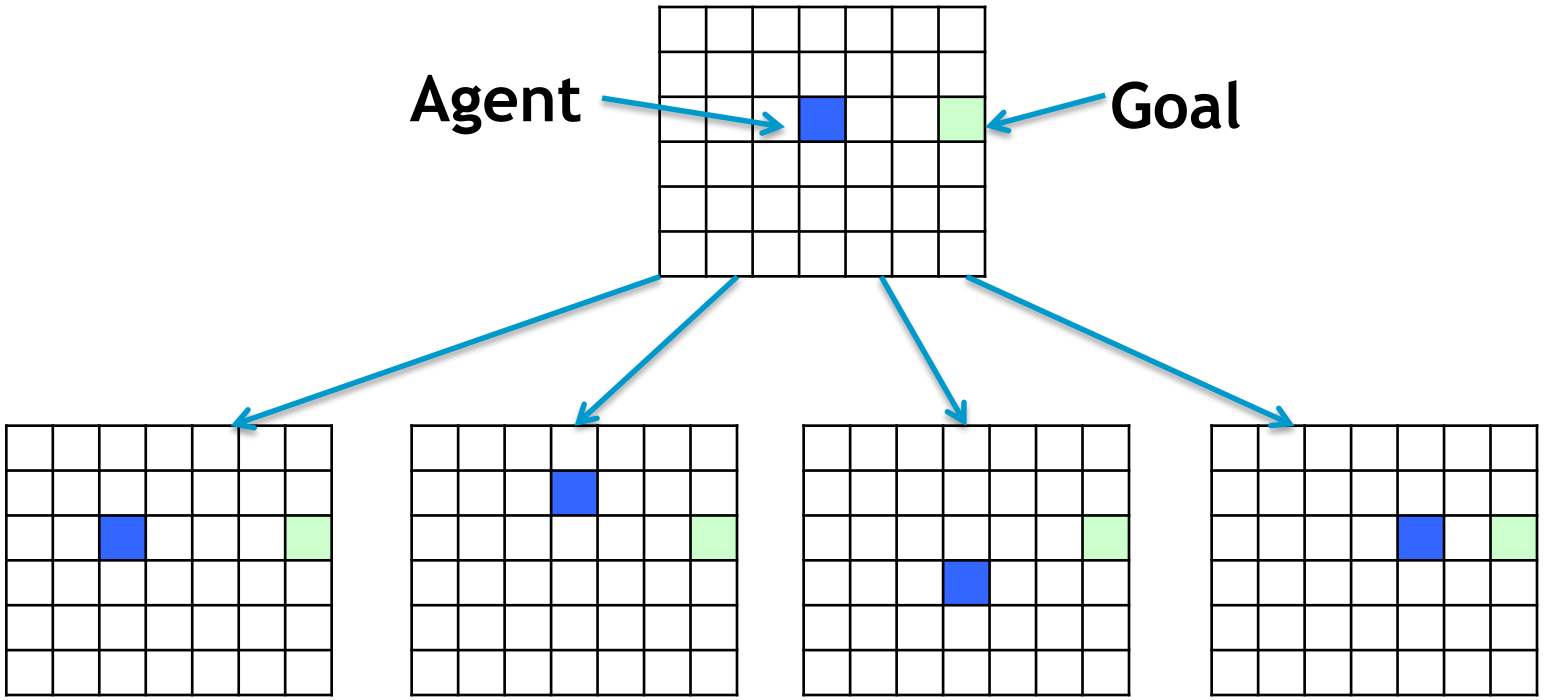
Example



There are 4 possible actions to execute.
Now, if you were the agent, which one would you try first?

Example

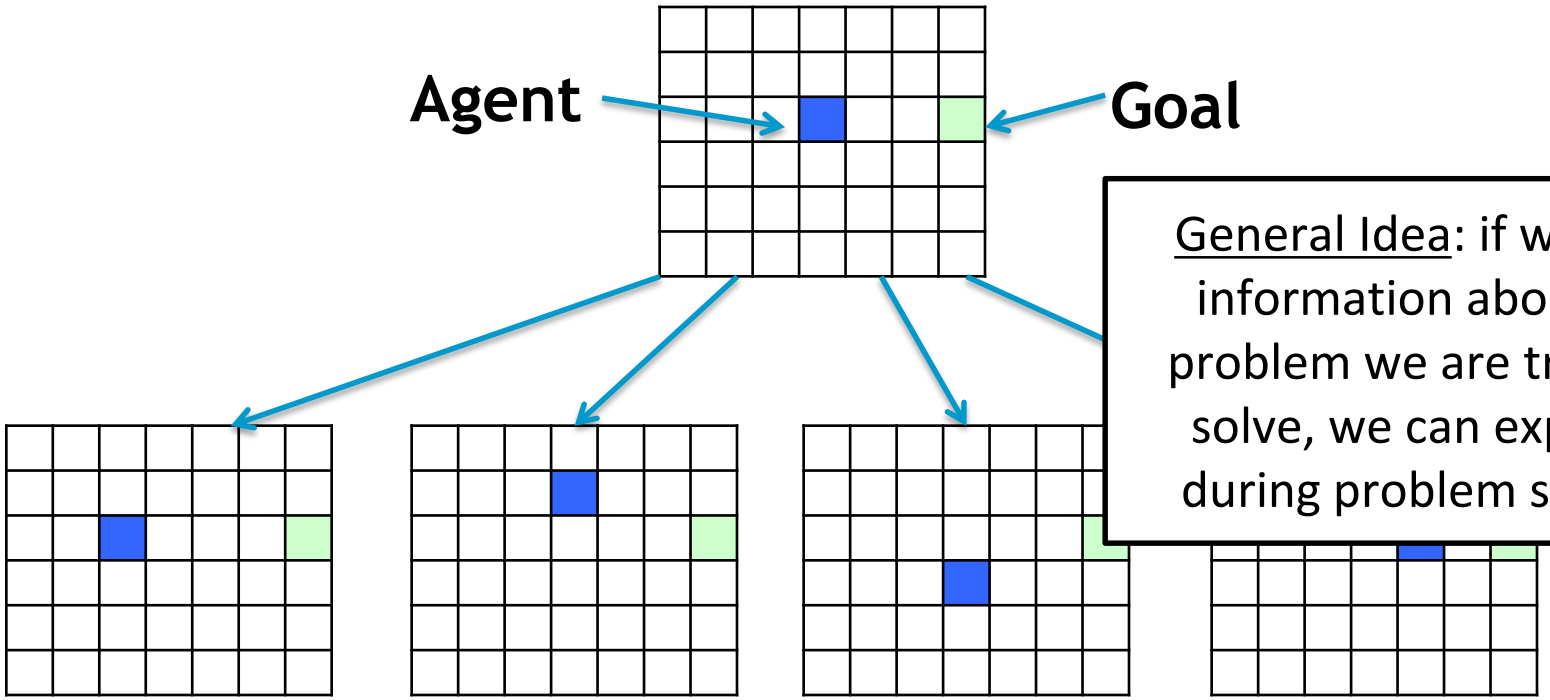
What if now you know what the coordinates mean?



There are 4 possible actions to execute.
Now, if you were the agent, which one would you try first?

Example

What if now you know what the coordinates mean?



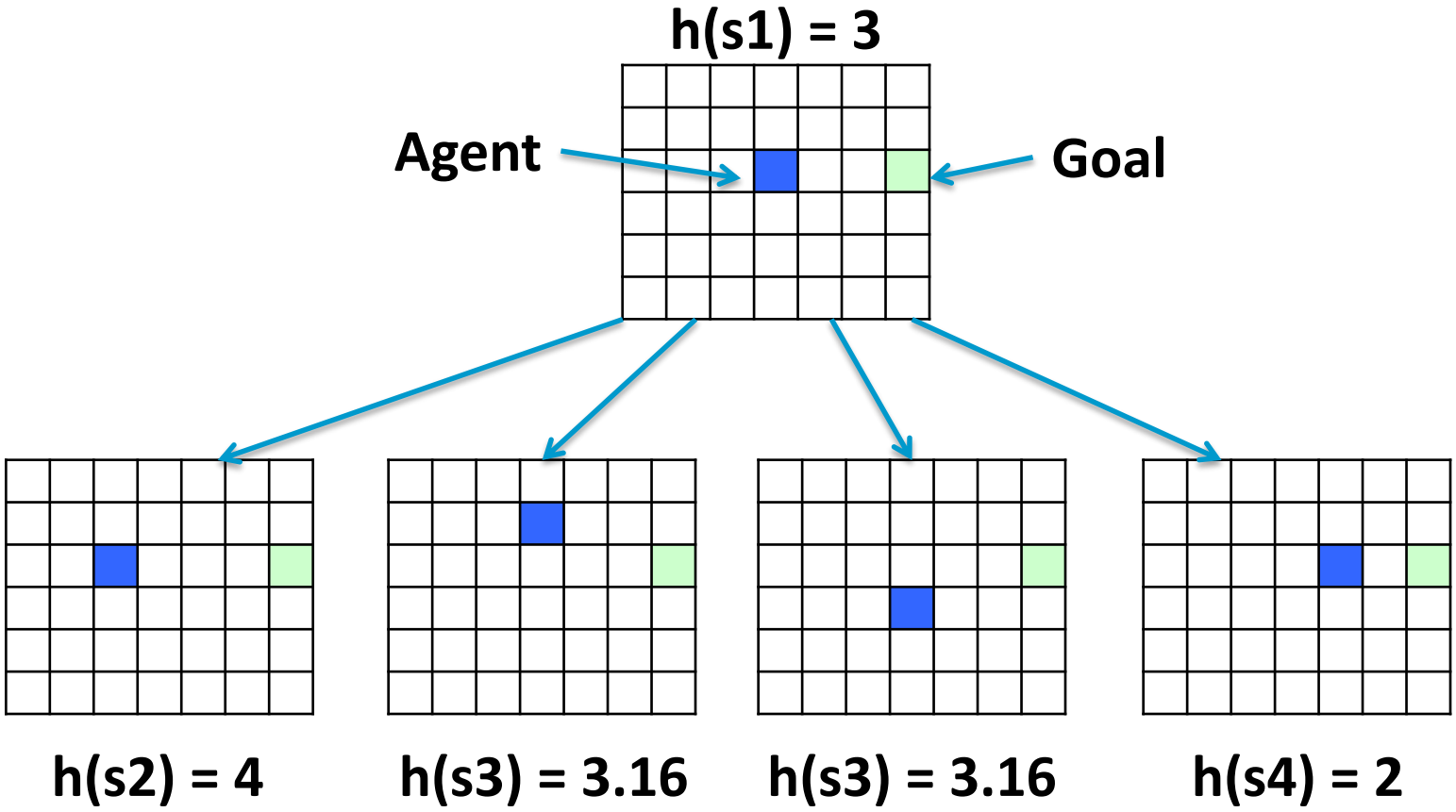
General Idea: if we have information about the problem we are trying to solve, we can exploit it during problem solving.

There are 4 possible actions to execute.
Now, if you were the agent, which one would you try first?

Evaluation Function

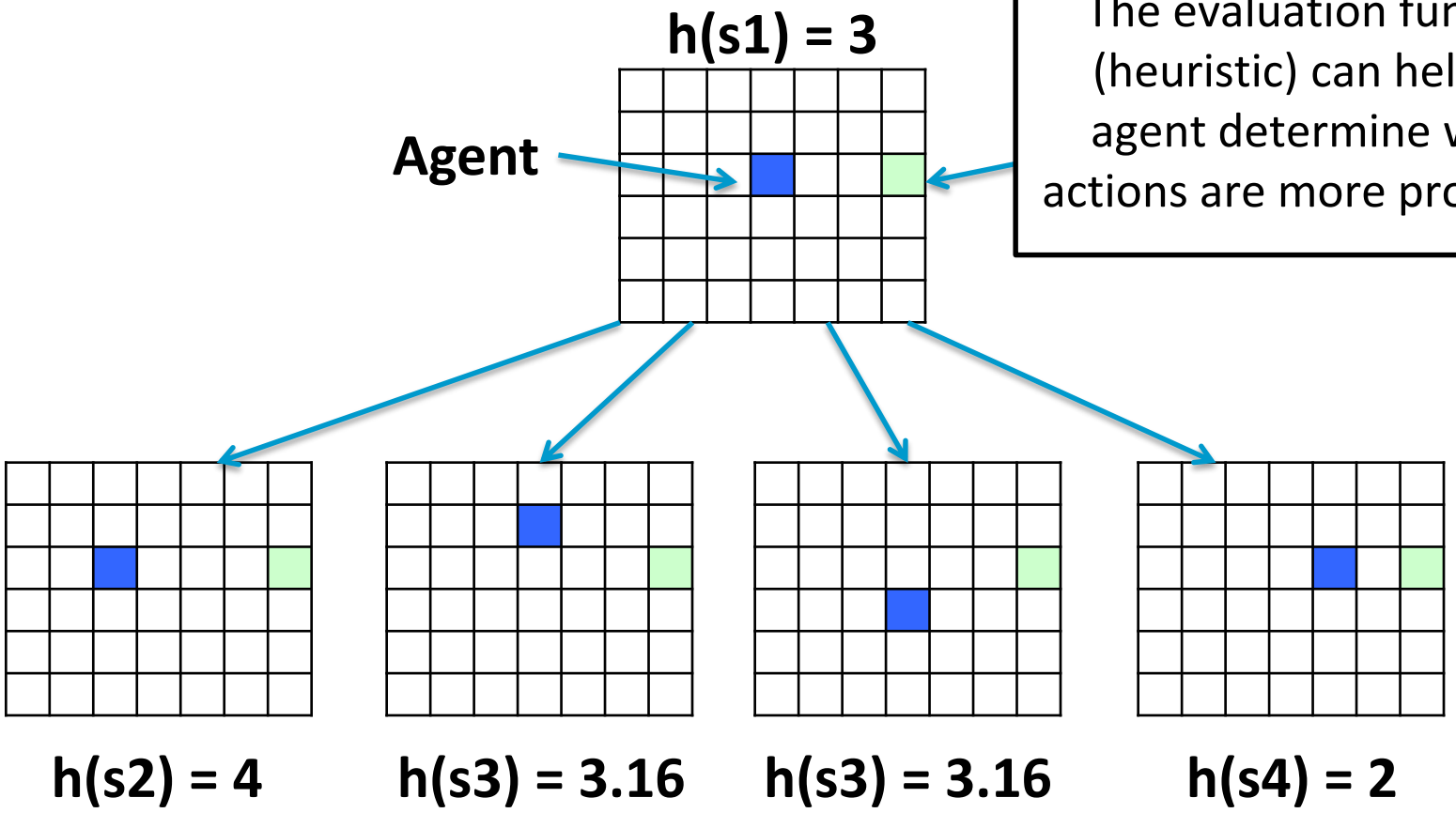
- Idea: represent the information we have about the domain as an "evaluation function" ***h***
- Evaluation function or "**heuristic**":
  - Given a state ***s*** …
  - ***h(s)*** estimates how far the state is from the goal
- For example, in a maze solving problem…
  - Euclidean distance to the goal?
  - Manhattan distance?
  - Etc.

Example



h: Euclidean distance to the goal

Example



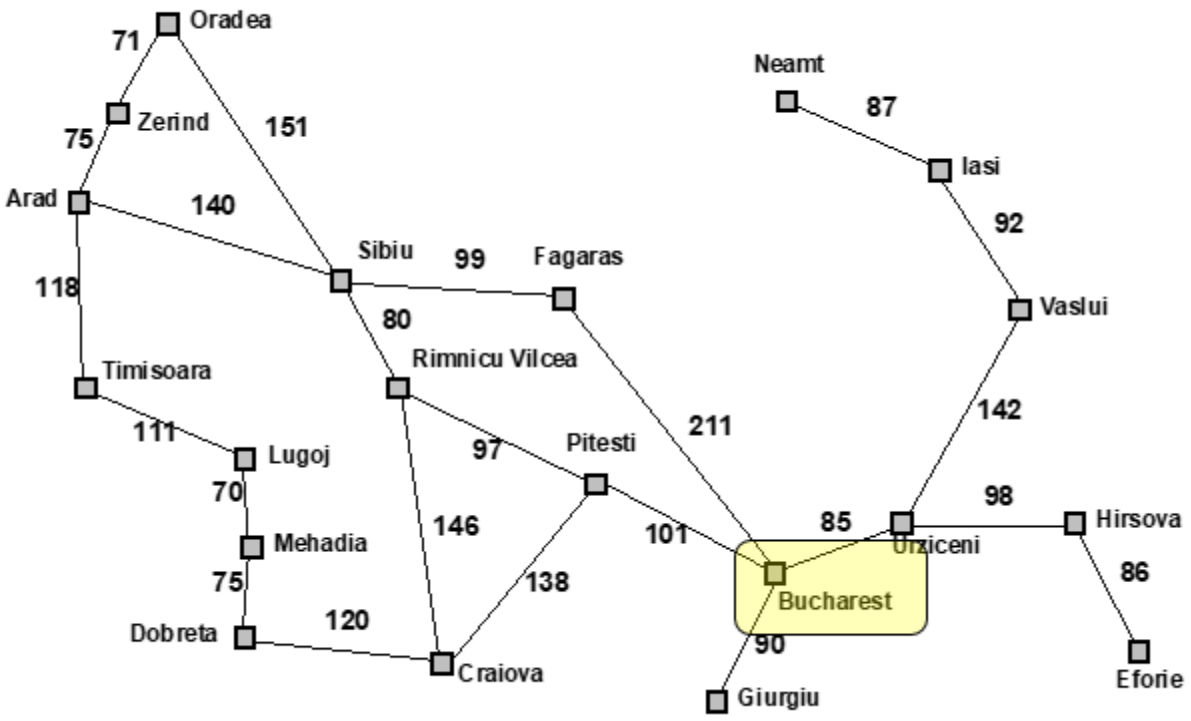
h: Euclidean distance to the goal

Best-first Search

- Idea: use an evaluation function for each node
  - estimate of "desirability"
  - expand most desirable unexpanded node
- Implementation:
  - fringe is a queue sorted in decreasing order of desirability
- Special cases:
  - Greedy search
  - A* search

Romania with step costs in km

Actual road connections and distances



Possible heuristic:

Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Search

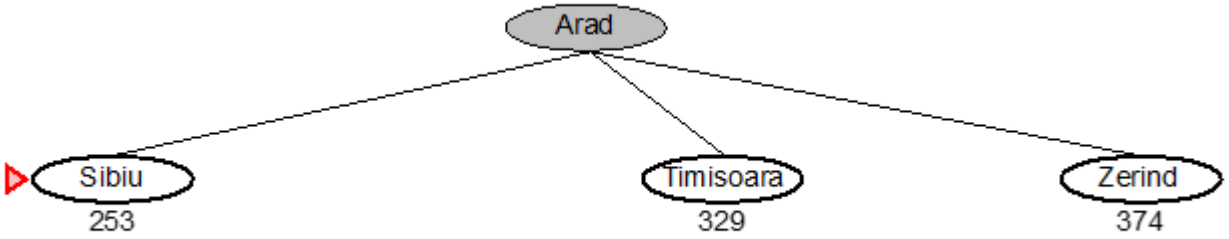
- Evaluation function h(n) (heuristic) = estimate of cost from n to the closest goal
- E.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest
- Greedy search expands the node that appears to be closest to goal

(What might go wrong with this heuristic?)

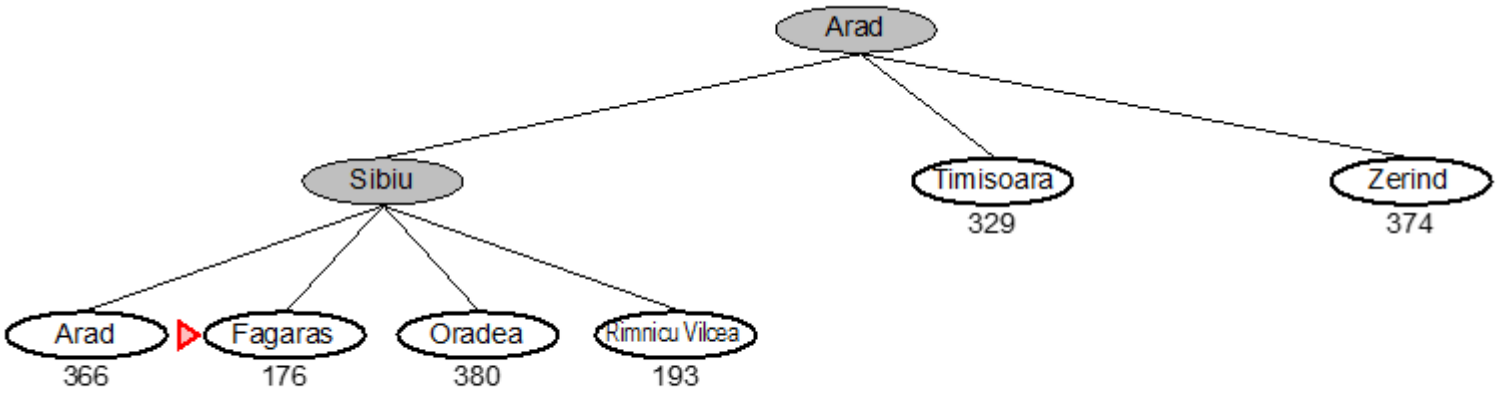
Greedy Search Example



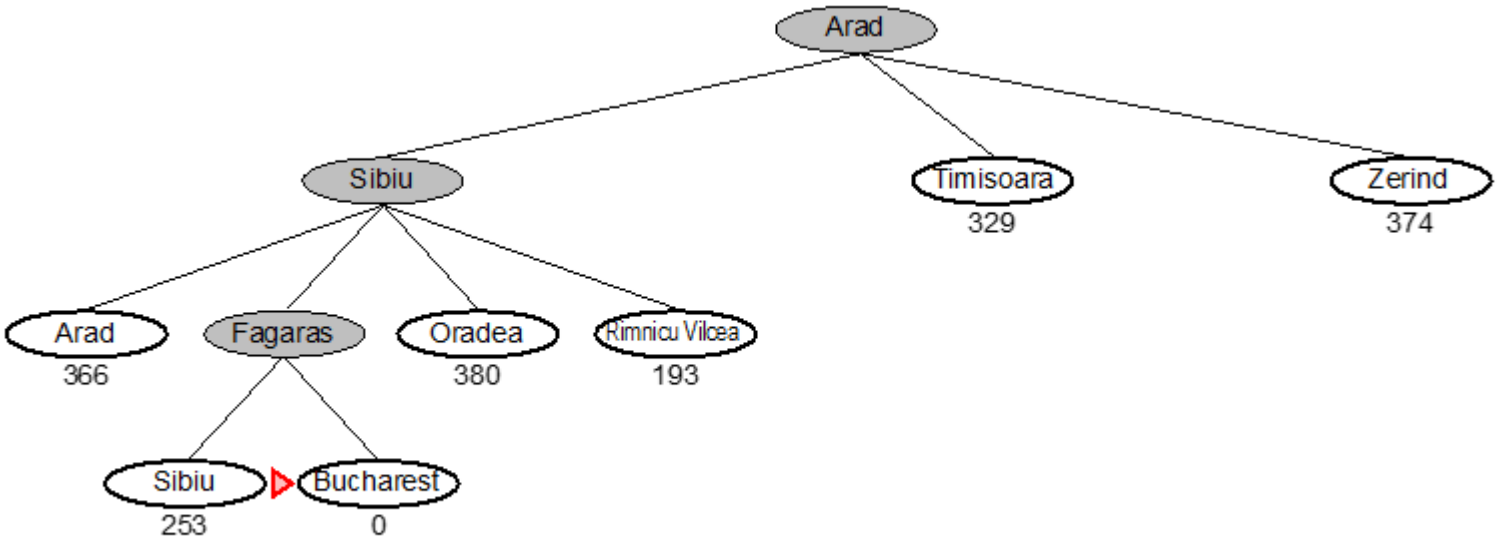
Greedy Search Example



Greedy Search Example



Greedy Search Example



Properties of Greedy Search

- Complete ??
- Time ??
- Space ??
- Optimal ??

Properties of Greedy Search

- Complete
  - No, it can get stuck in loops, e.g., with Oradea as goal, Iasi -> Neamt -> Iasi -> Neamt -> …
  - Complete in finite space with repeated-state checking
- Time ??
- Space ??
- Optimal ??

Properties of Greedy Search

- Complete
  - No, it can get stuck in loops, e.g., with Oradea as goal, lasi -> Neamt -> lasi -> Neamt -> …
  - Complete in finite space with repeated-state checking
- Time
  - $O(b^m)$, but a good heuristic can give dramatic improvement
- Space ??
- Optimal ??

Consider
m = maximum depth

Properties of Greedy Search

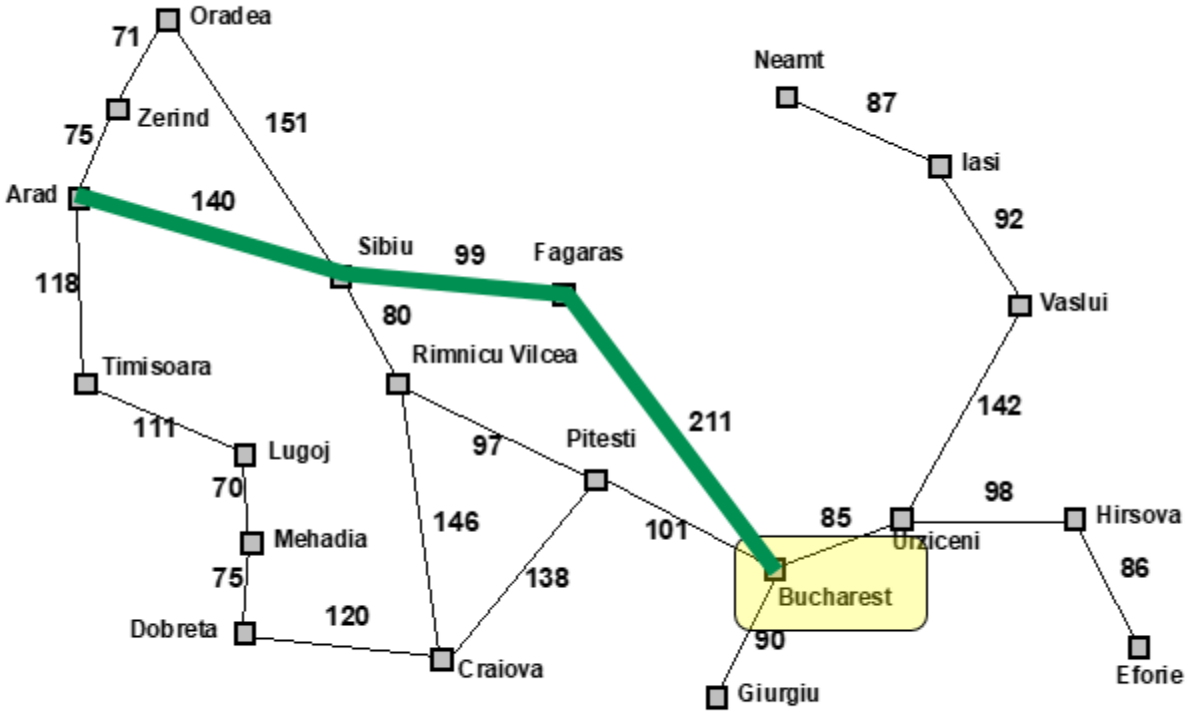
- Complete
  - No, it can get stuck in loops, e.g., with Oradea as goal, lasi -> Neamt -> lasi -> Neamt -> …
  - Complete in finite space with repeated-state checking
- Time
  - $O(b^m)$, but a good heuristic can give dramatic improvement
- Space
  - $O(b^m)$ keeps all nodes in memory
- Optimal ??

Properties of Greedy Search

- Complete
  - No, it can get stuck in loops, e.g., with Oradea as goal, lasi -> Neamt -> lasi -> Neamt -> …
  - Complete in finite space with repeated-state checking
- Time
  - $O(b^m)$, but a good heuristic can give dramatic improvement
- Space
  - $O(b^m)$ keeps all nodes in memory
- Optimal
  - No

Romania with Step Costs in km

Actual connections and distances



Possible heuristic:

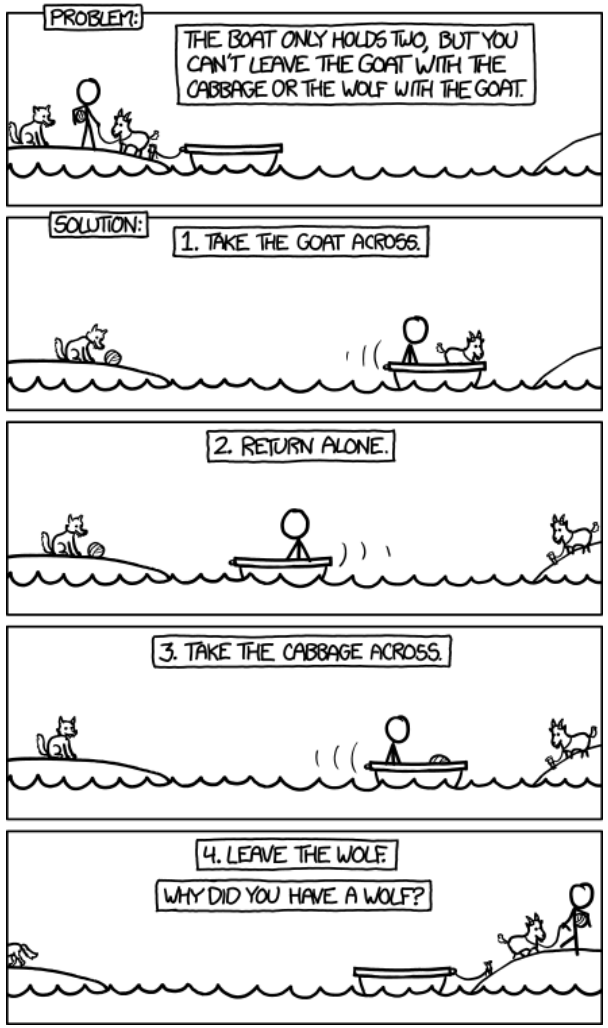
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Is this path optimal?

Heuristics

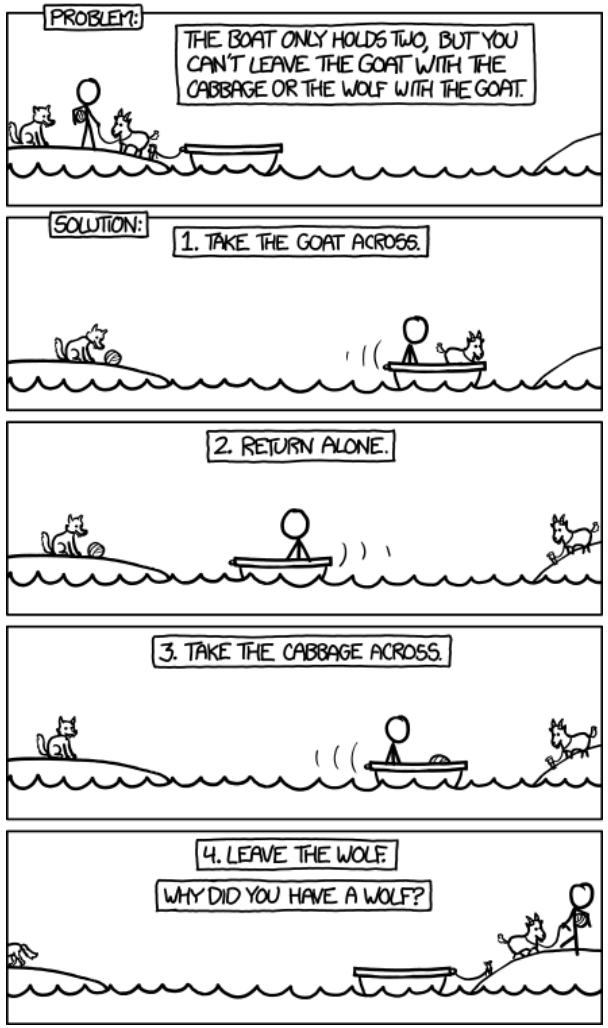
- Idea: Estimate the distance to the solution
- Must be computationally efficient (otherwise, one could just run the complete search algorithm to determine the actual cost!)
- Finding good heuristics makes a huge difference in computation time

Heuristics: Wolf-Goat-Cabbage



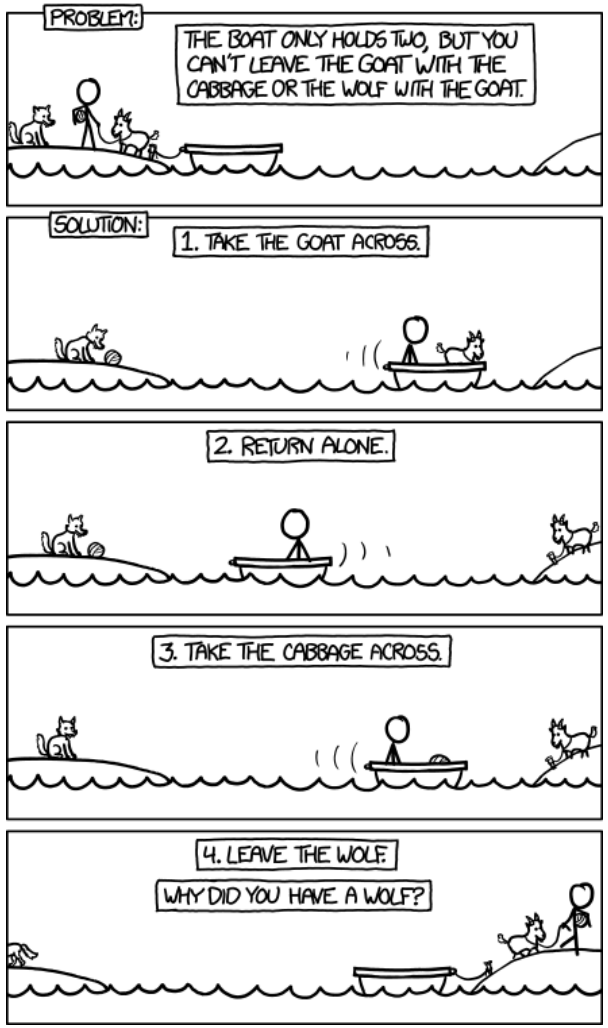
- Let's assume that "cost" is the number of boat trips.
- What would be a good heuristic?

Heuristics: Wolf-Goat-Cabbage



- Let's assume that "cost" is the number of boat trips.
- What would be a good heuristic?
- Example:
  - Number of "items" on the left bank (since we know we need to make at least that number of trips)

Heuristics: Wolf-Goat-Cabbage



- Let's assume that "cost" is the number of boat trips.
- What would be a good heuristic?
- Example:
  - Number of "items" on the left bank (since we know we need to make at least that number of trips)
  - Better:
    - ([*number of items in left bank*] – 1) * 2
    - Add 1 if boat on right bank and [*number of items in left bank*] > 0

Heuristics: 8-Puzzle

8		1
3	2	6
7	4	5

- Cost: number of moves
- What would be a good heuristic?

Heuristics: 8-Puzzle

8		1
3	2	6
7	4	5

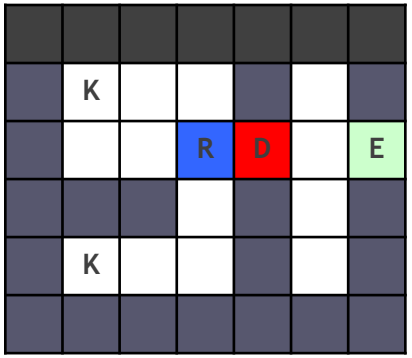
- Cost: number of moves
- What would be a good heuristic?
  - Heuristic 1:
    - Number of blocks out of place

Heuristics: 8-Puzzle

8		1
3	2	6
7	4	5

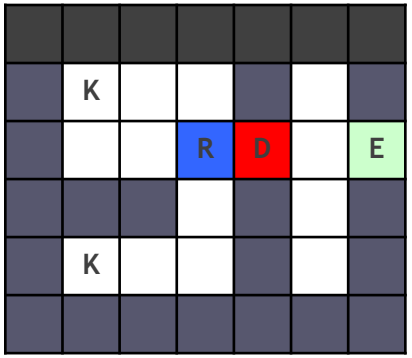
- Cost: number of moves
- What would be a good heuristic?
  - Heuristic 1:
    - Number of blocks out of place
  - Heuristic 2:
    - Distance of each block to its target position

Heuristic: Robot navigation with keys



- A robot (blue) needs to exit a maze (light green), and needs to pick up keys to cross doors (red)

Heuristic: Robot navigation with keys



- A robot (blue) needs to exit a maze (light green), and needs to pick up keys to cross doors (red)
- Heuristic 1:
  - Simply Euclidean distance to goal (ignoring keys and doors)
  - Can you improve it?

A Note on Graph Search

- Repeated-state checking:
  - When the search state is a graph, strategies like DFS can get stuck in loops
  - To avoid this, we generally want to keep a list of already-visited nodes – a "CLOSED" list
- In DFS:
  - If we want to avoid repeating states completely, we need to keep ALL visited states in memory (in the CLOSED list)
  - If we just want to avoid loops, we only need to remember the current branch (linear memory as a function of *m* = maximum depth)

A* Search

- Pronounced “A-Star”
- At each cycle, expand the node with the lowest *f(n)*

$$f(n) = g(n) + h(n)$$

Real cost from Start to n

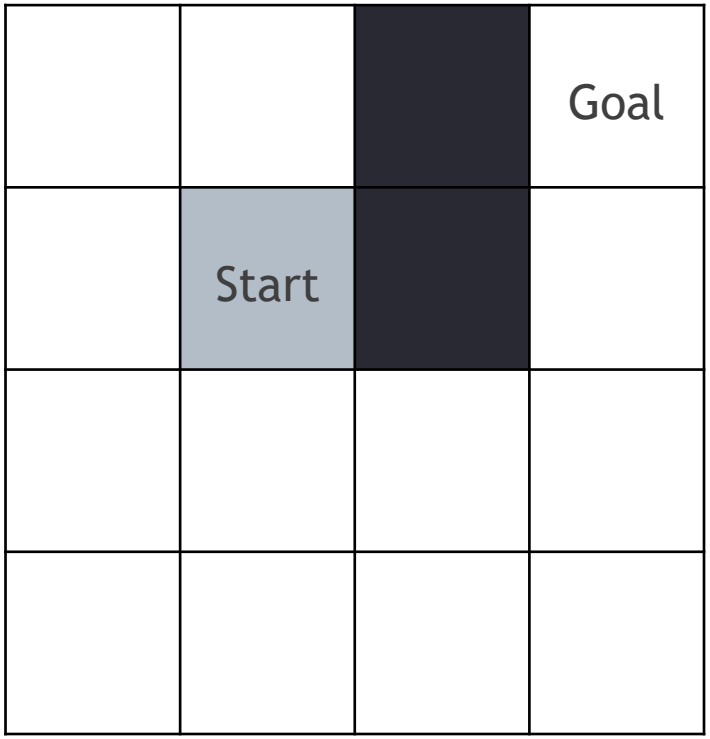
Heuristic

- A* implementations assume repeated-state checking (i.e., assume the search space is a graph)
- Thus, they use two lists simultaneously:
  - OPEN: list of nodes that need to be processed
  - CLOSED: list of nodes that have already been processed

Example: A*



OPEN = [Start]
CLOSED = []

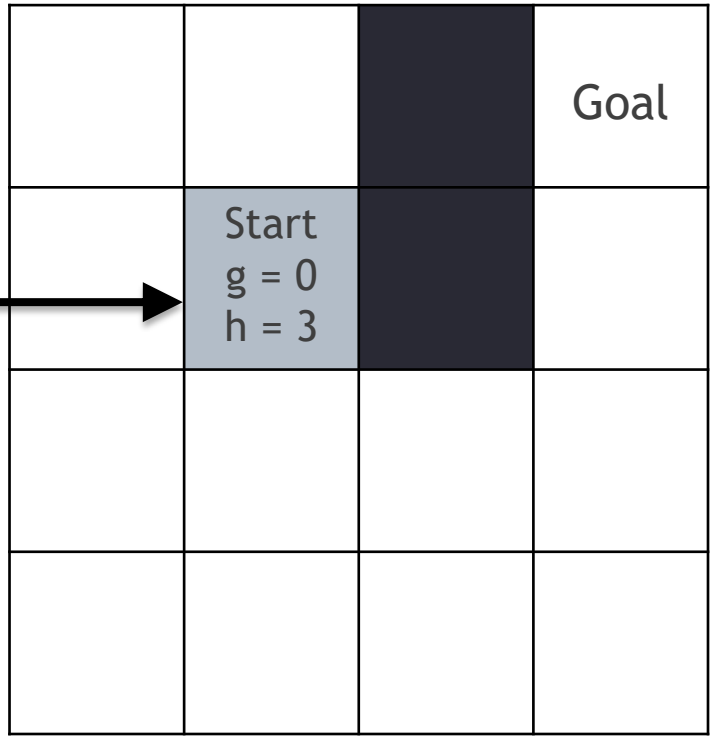


Heuristic used: **Manhattan Distance**

Example: A*



OPEN = [Start]
CLOSED = []



Assigns an "estimated cost", *f*, to each node:

$f(n) = g(n) + h(n)$

Real cost from Start to n

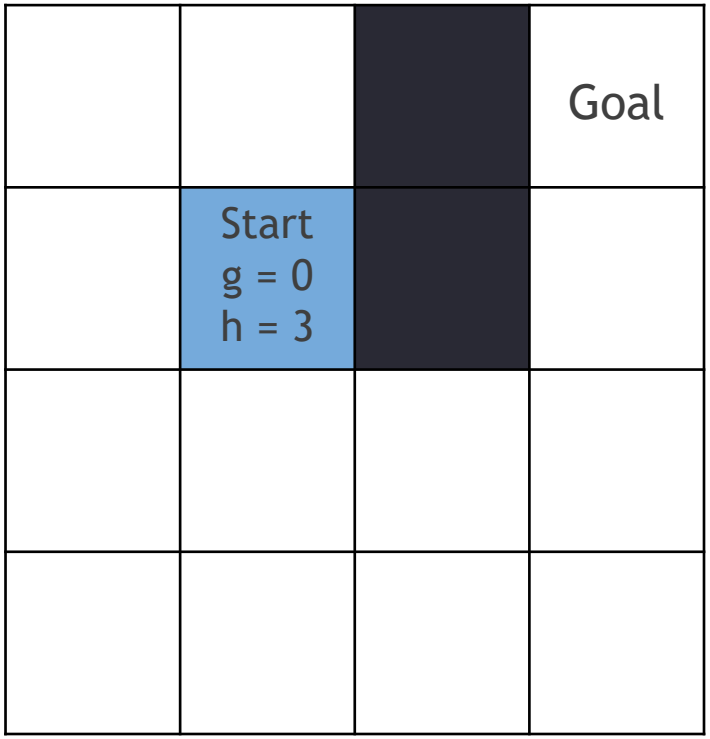
Heuristic

Heuristic used: **Manhattan Distance**

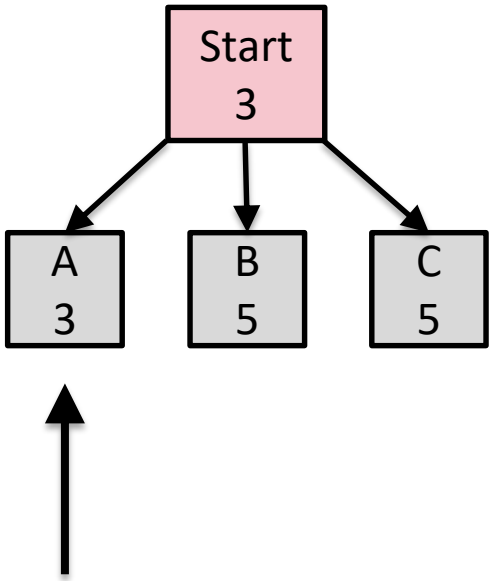
Example: A*

Start
3

OPEN = [Start]
CLOSED = []



Example: A*

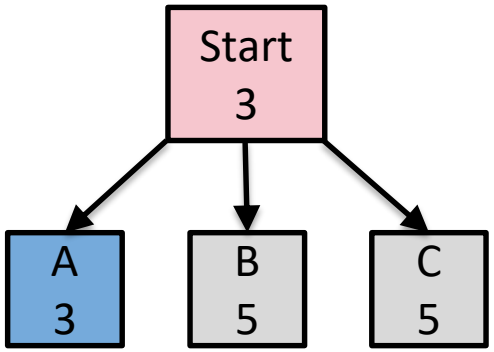


Expands the node with the lowest estimated cost first

OPEN = [A, B, C]
CLOSED = [Start]

	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
	C g = 1 h = 4		

Example: A*

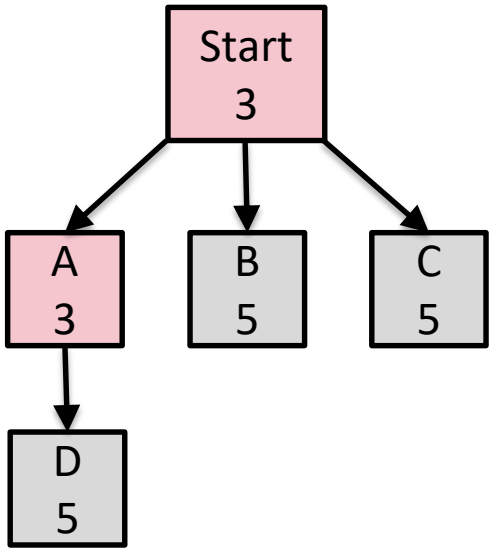


Expands the node with the lowest estimated cost first

OPEN = [A, B, C]
CLOSED = [Start]

	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
	C g = 1 h = 4		

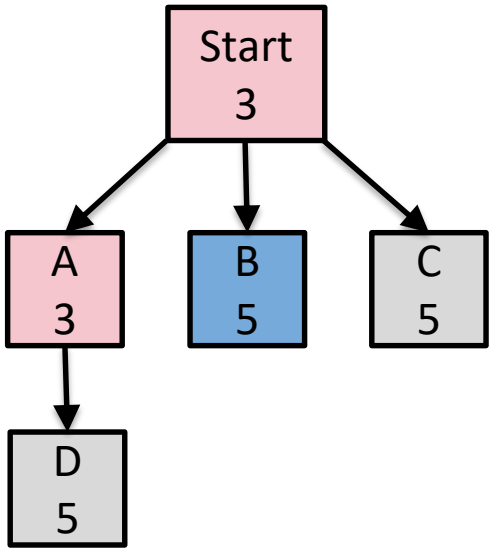
Example: A*



OPEN = [B, C, D]
CLOSED = [Start, A]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
	C g = 1 h = 4		

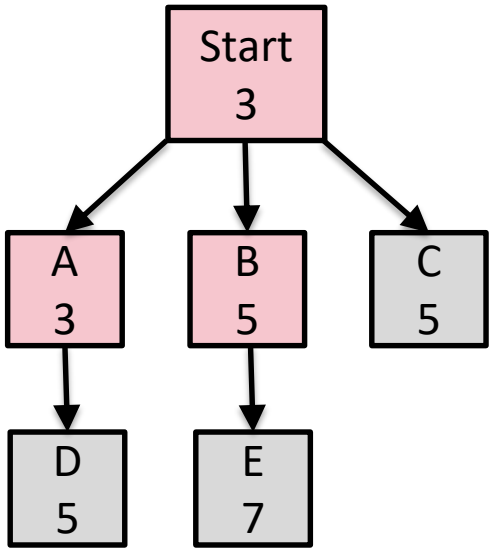
Example: A*



OPEN = [B, C, D]
CLOSED = [Start, A]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
	C g = 1 h = 4		

Example: A*

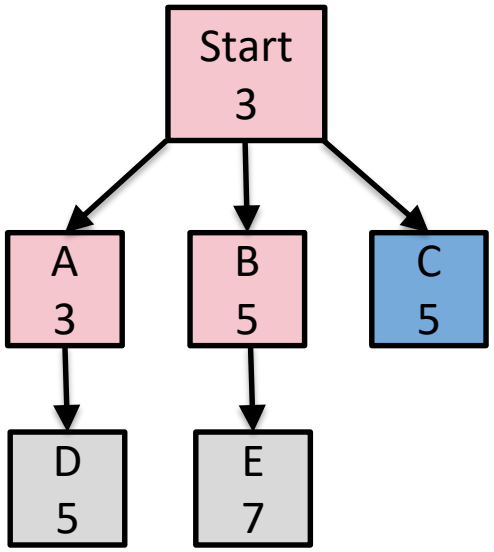


OPEN = [C, D, E]

CLOSED = [Start, A, B]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4		

Example: A*

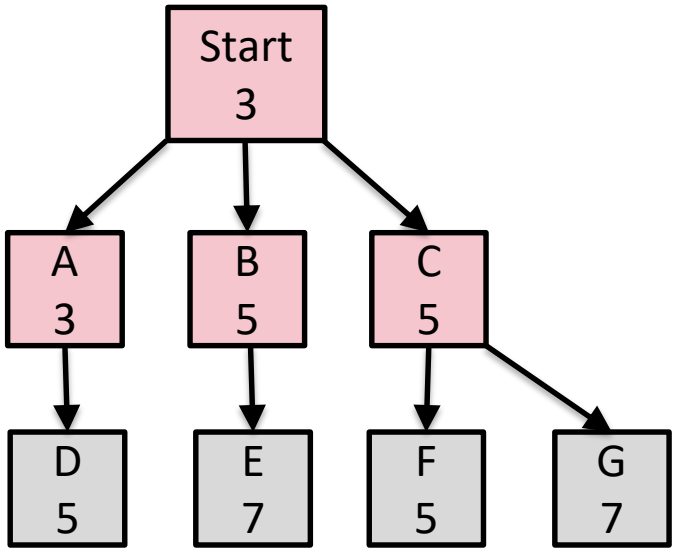


OPEN = [C, D, E]

CLOSED = [Start, A, B]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4		

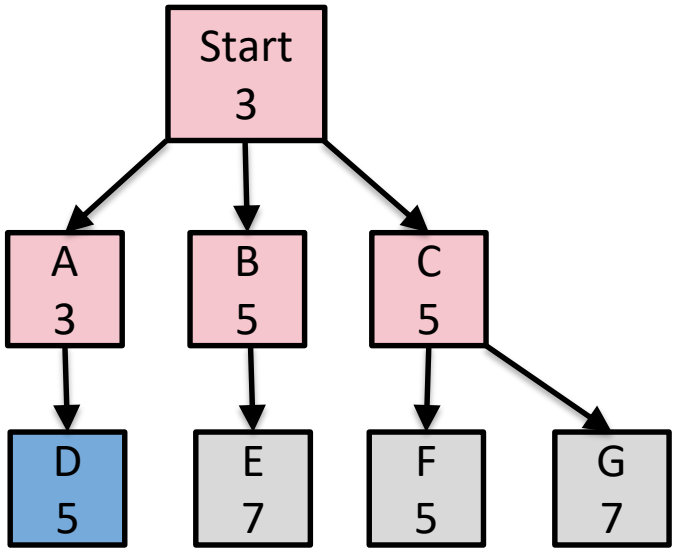
Example: A*



OPEN = [D, F, E, G]
CLOSED = [Start, A, B, C]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	
	G g = 2 h = 5		

Example: A*

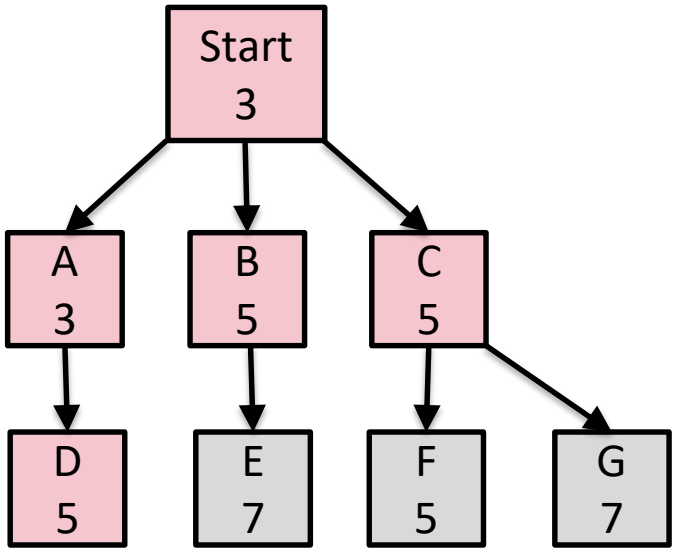


OPEN = [D, F, E, G]

CLOSED = [Start, A, B, C]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	
	G g = 2 h = 5		

Example: A*

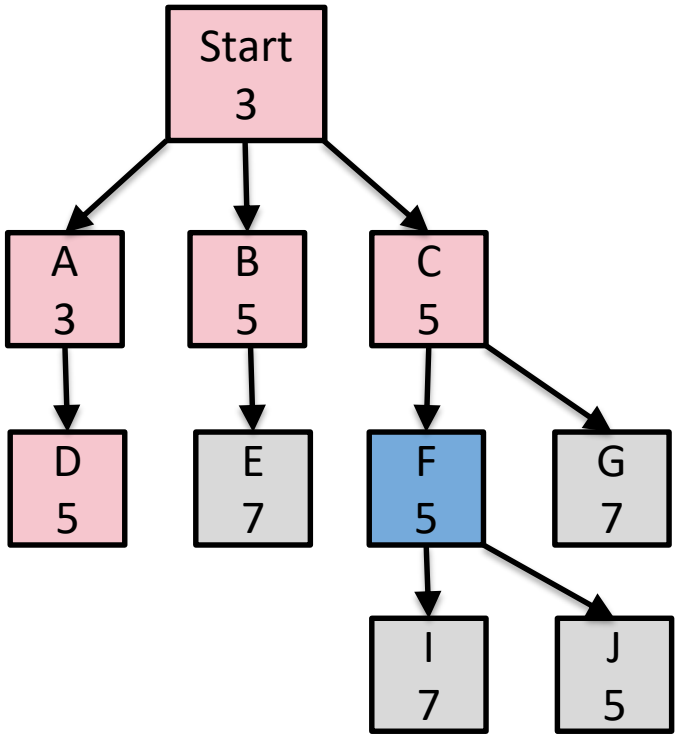


OPEN = [F, E, G]

CLOSED = [Start, A, B, C, D]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	
	G g = 2 h = 5		

Example: A*

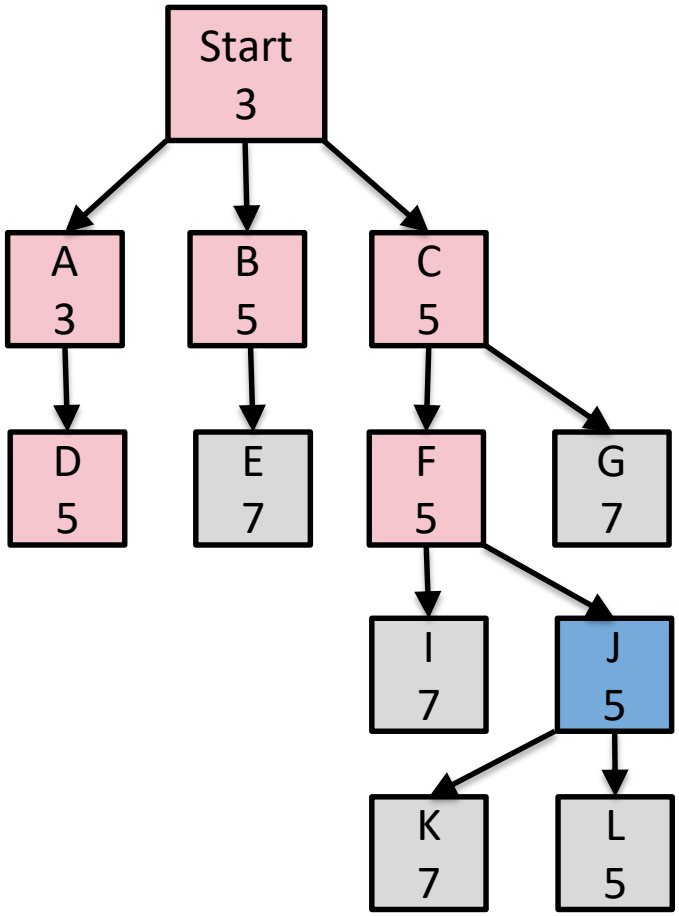


OPEN = [J, E, G, I]

CLOSED = [Start, A, B, C, D, F]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	

Example: A*

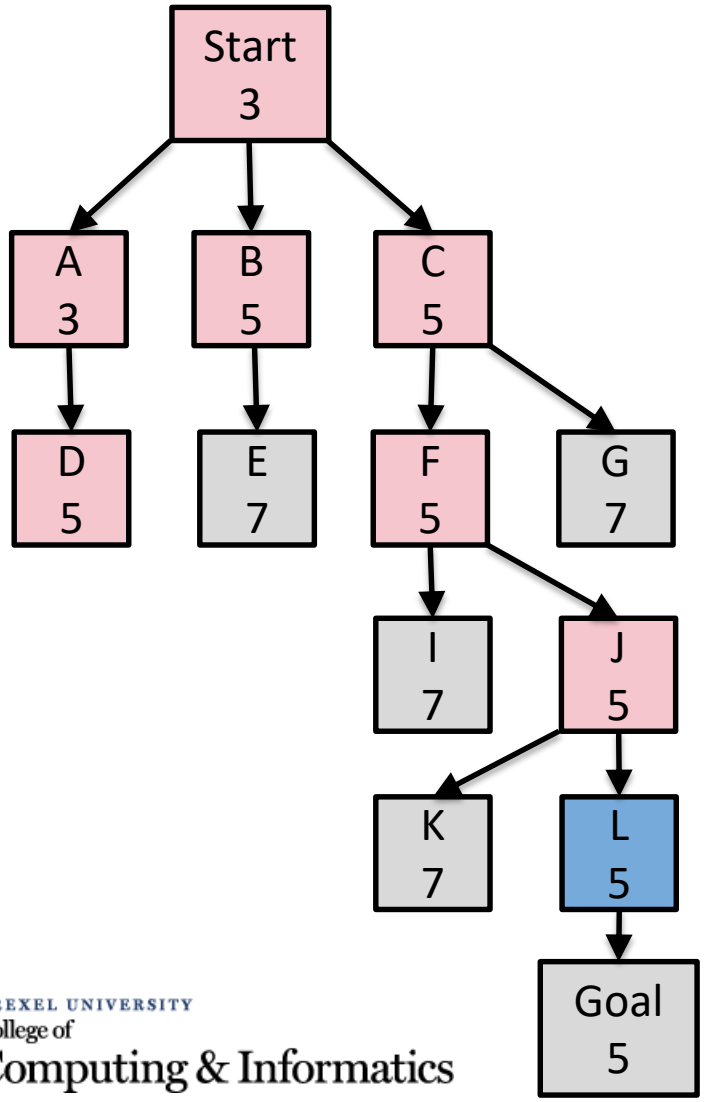


OPEN = [L, E, G, I, K]

CLOSED = [Start, A, B, C, D, F, J]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

Example: A*

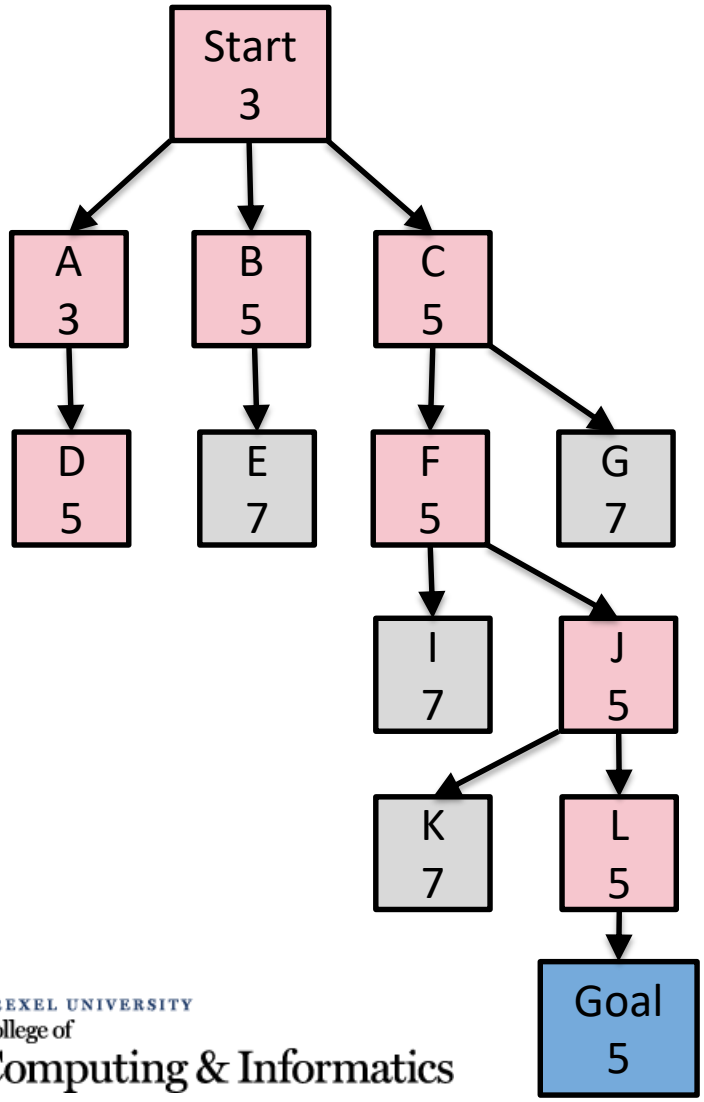


OPEN = [Goal, E, G, I, K]

CLOSED = [Start, A, B, C, D, F, J, L]

D g = 2 h = 3	A g = 1 h = 2		Goal g = 5 h = 0
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

Example: A*



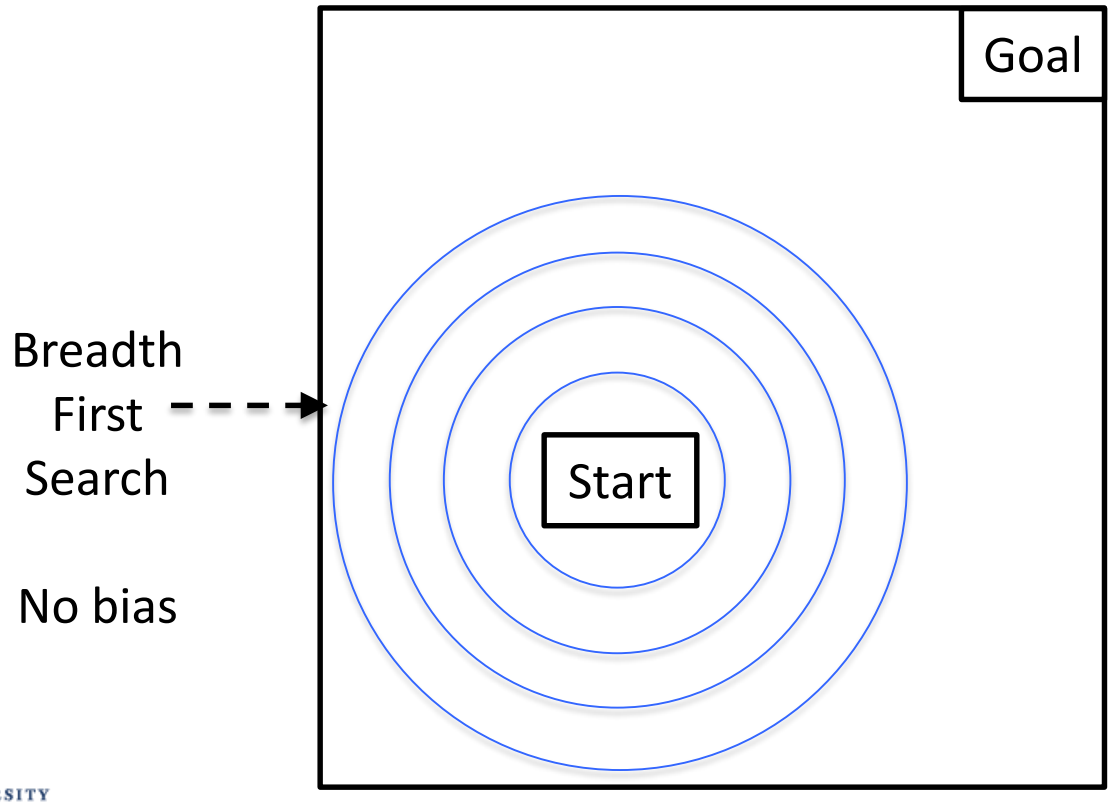
OPEN = [E, G, I, K]

CLOSED = [Start, A, B, C, D, F, J, L, Goal]

D g = 2 h = 3	A g = 1 h = 2		Goal g = 5 h = 0
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

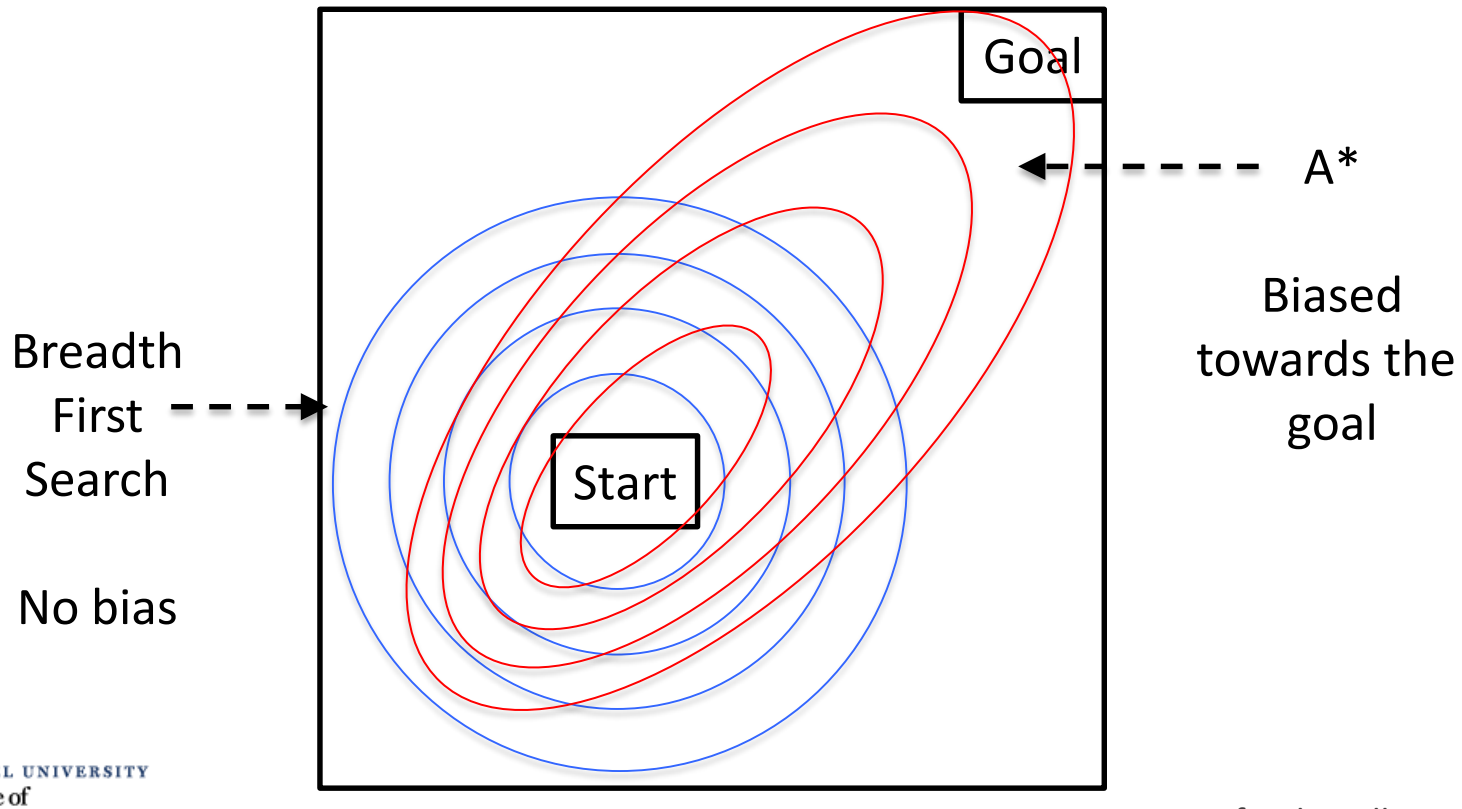
A* Intuition

- The heuristic biases the search of the algorithm towards the goal:

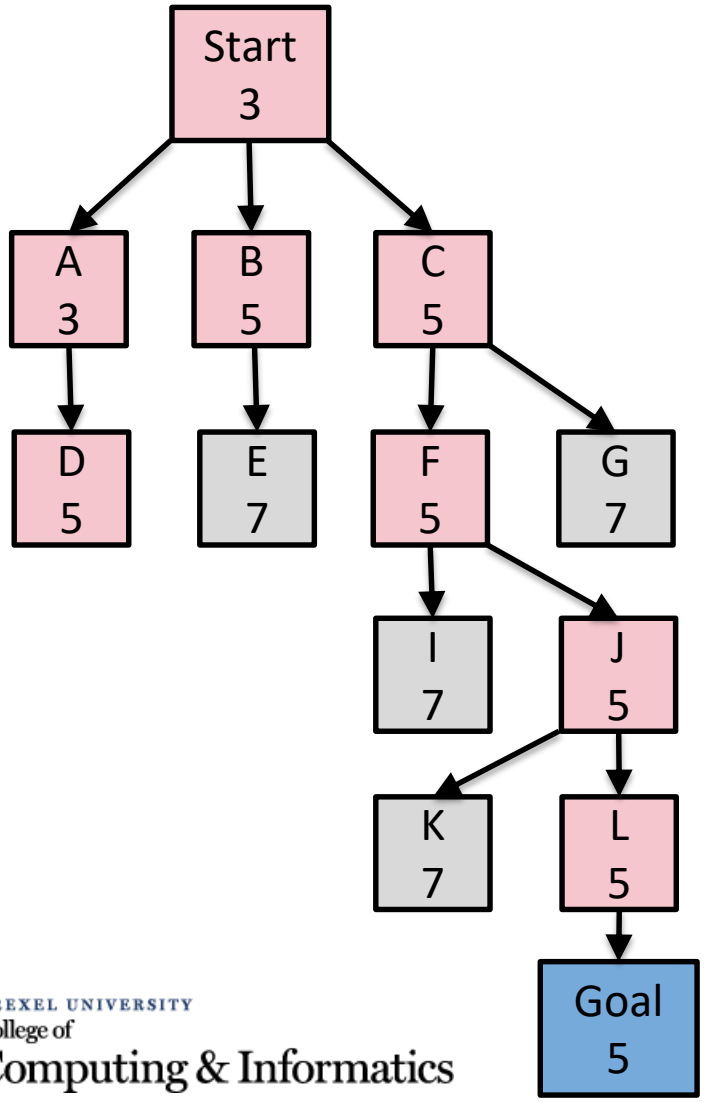


A* Intuition

- The heuristic biases the search of the algorithm towards the goal:



Example: A*



OPEN = [E, G, I, K]

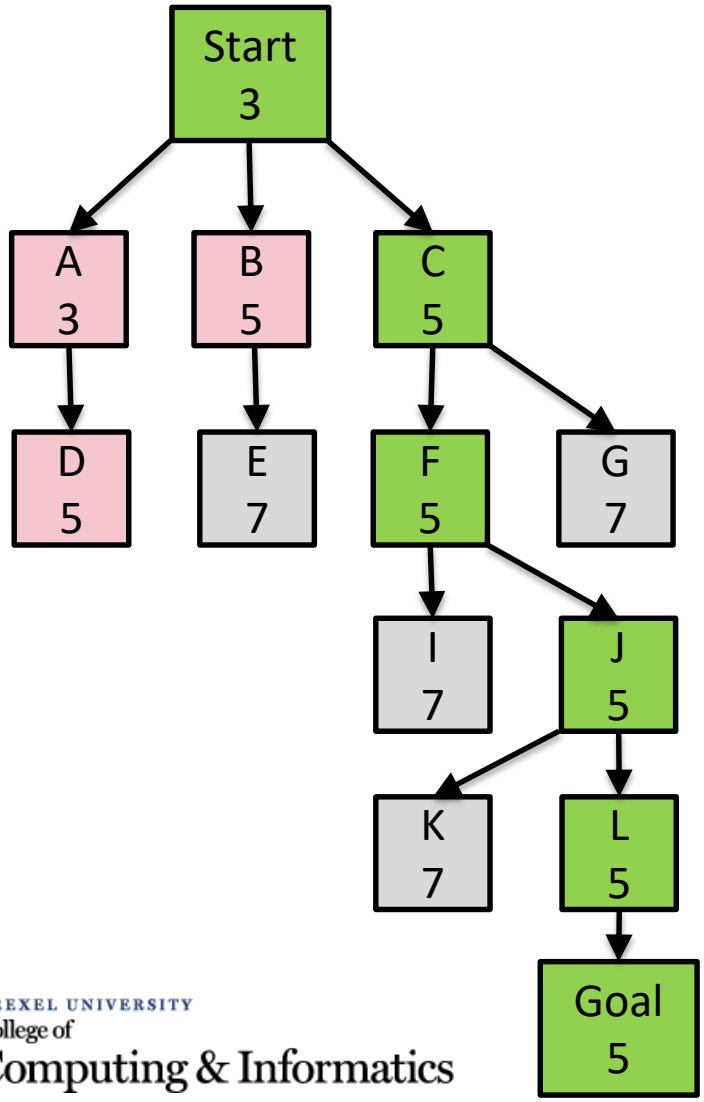
CLOSED = [Start, A, B, C, D, F, J, L, Goal]

D g = 2 h = 3	A g = 1 h = 2		Goal g = 5 h = 0
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

Implementation Notes

- Remember the distinction between **state** and **node**
- State:
  - The configuration of the problem (e.g., coordinates of a robot, positions of the pieces in the 8-puzzle, etc.)
- Node:
  - State + {current cost (g), current heuristic (h), parent node, action that got us here from the parent node, …}
  - It is important to remember who was the parent, and which action was applied, so that once the solution is found, we can reconstruct the path

Example: A*



OPEN = [E, G, I, K]

CLOSED = [Start, A, B, C, D, F, J, L, Goal]

D g = 2 h = 3	A g = 1 h = 2		Goal g = 5 h = 0
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

A* Pseudocode

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

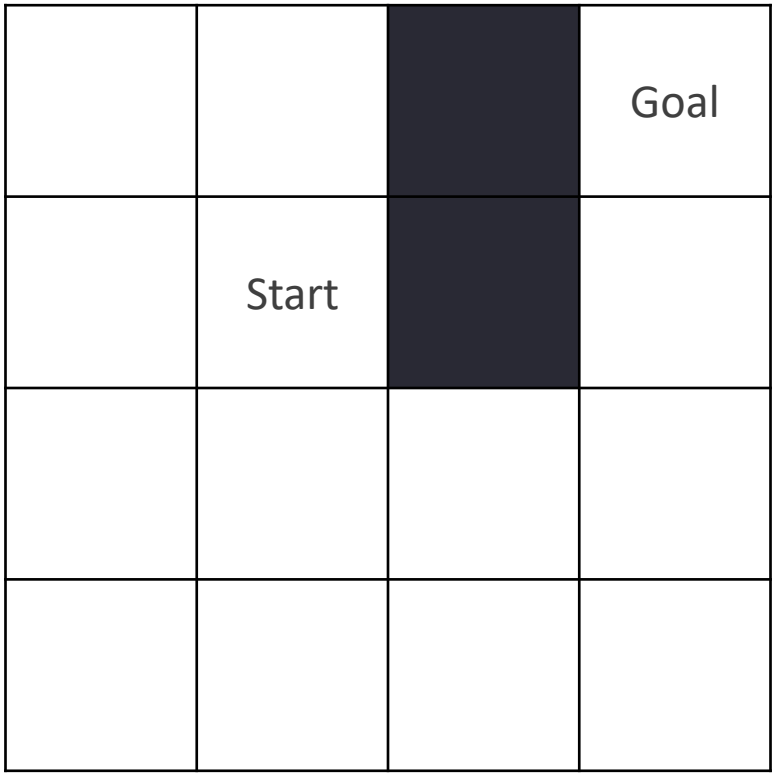
A* Pseudocode

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
    CLOSED.add(N)
    FOR all children M of N not in CLOSED:
            M.parent = N
            M.g = N.g + 1
            M.h = heuristic(M)
            OPEN.add(M)
    ENDFOR
ENDWHILE
```

Differences with respect to Breadth First Search

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

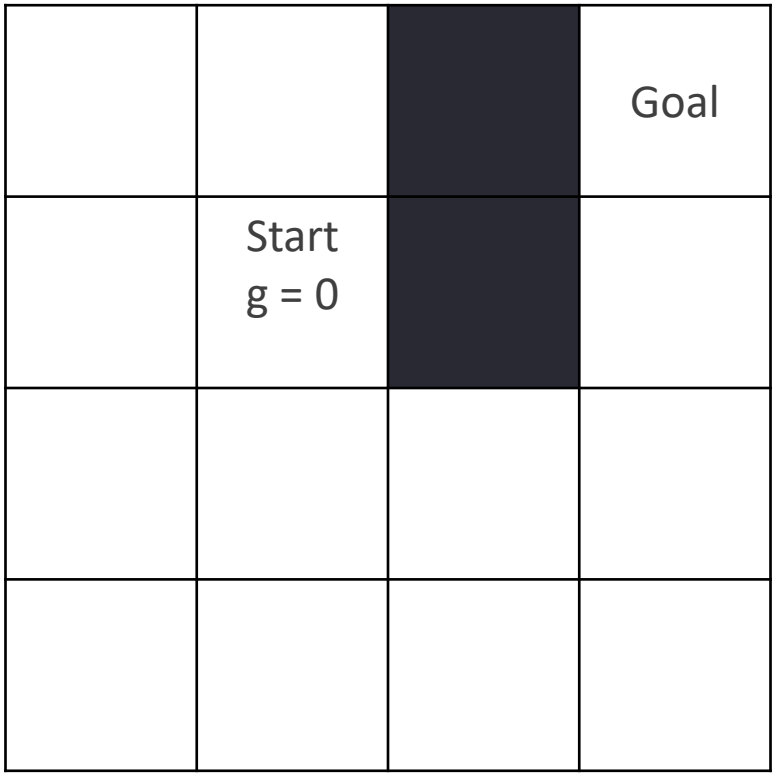


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

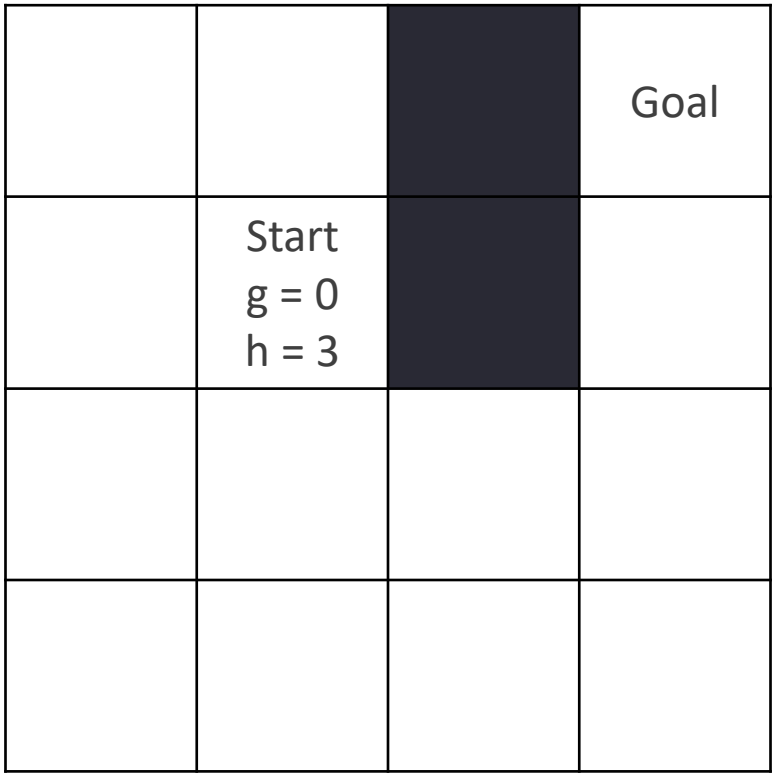


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

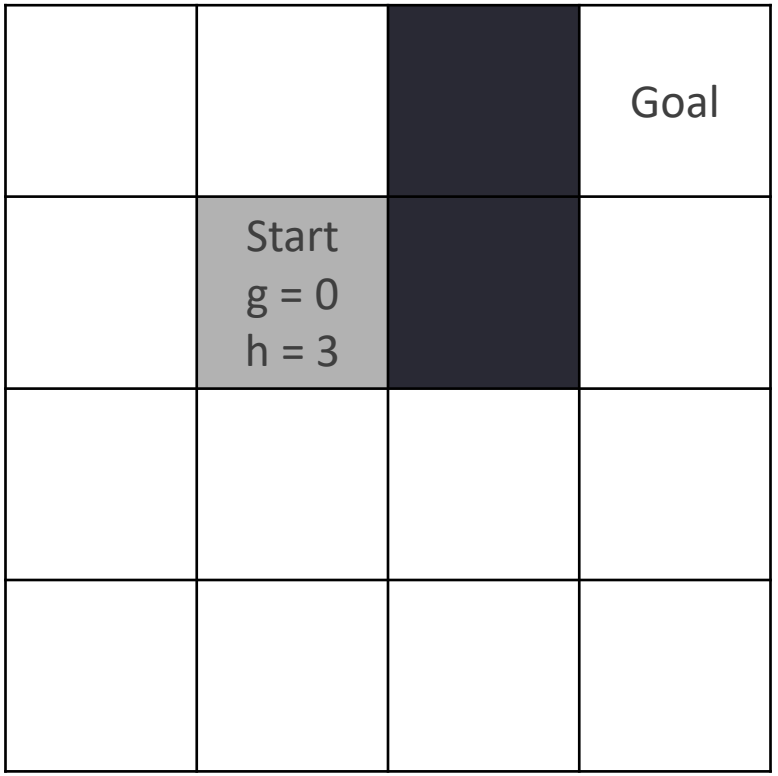


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

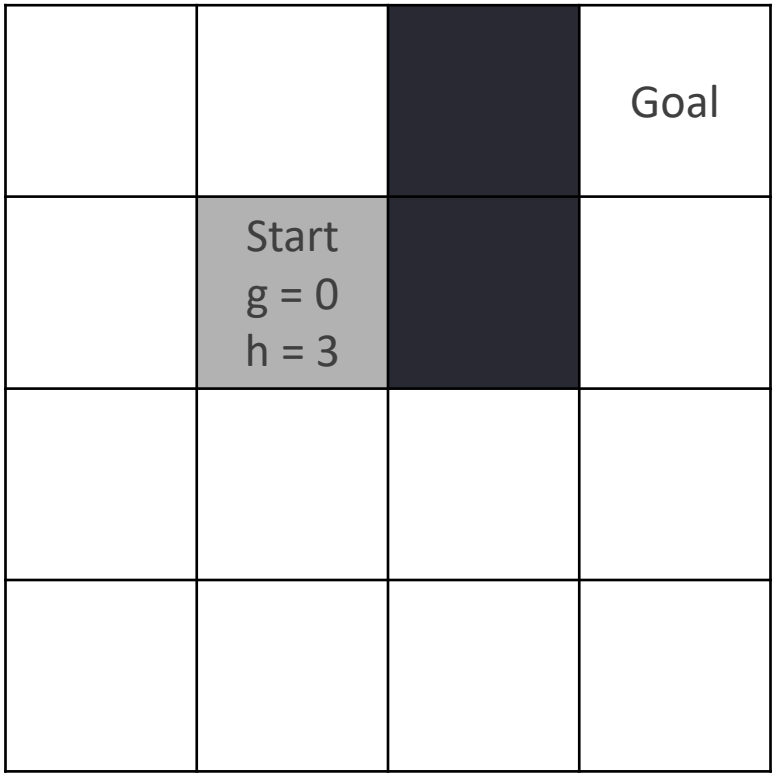


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

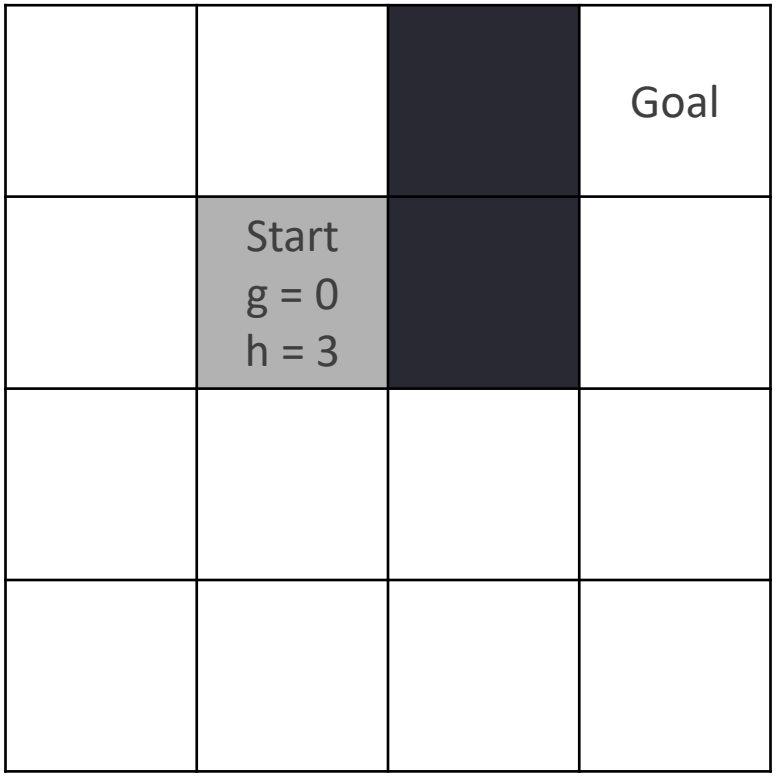


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

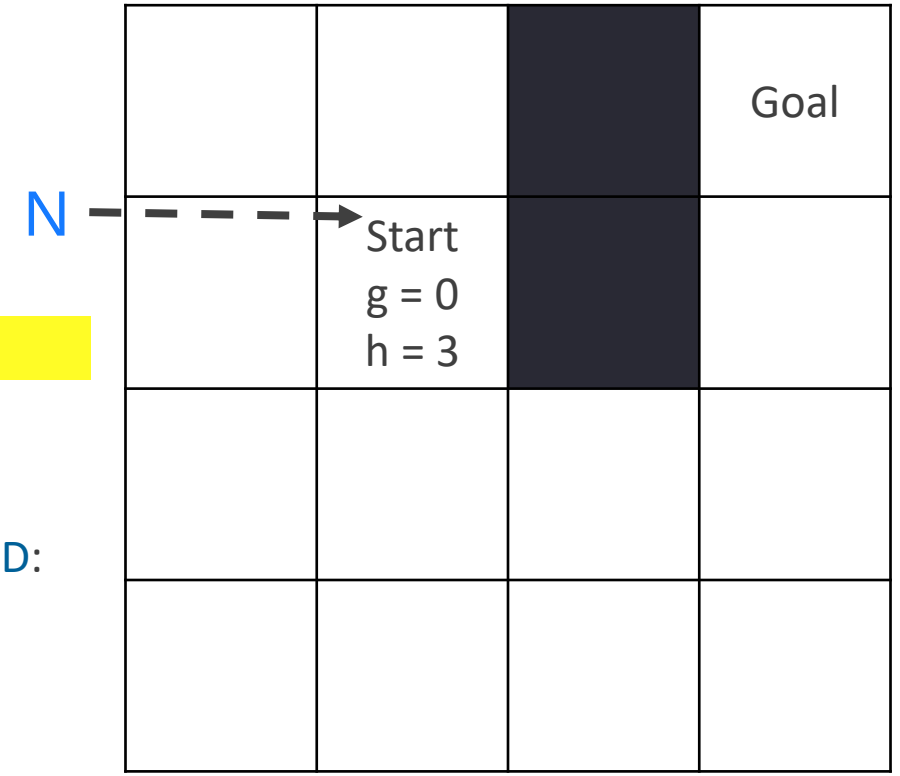


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

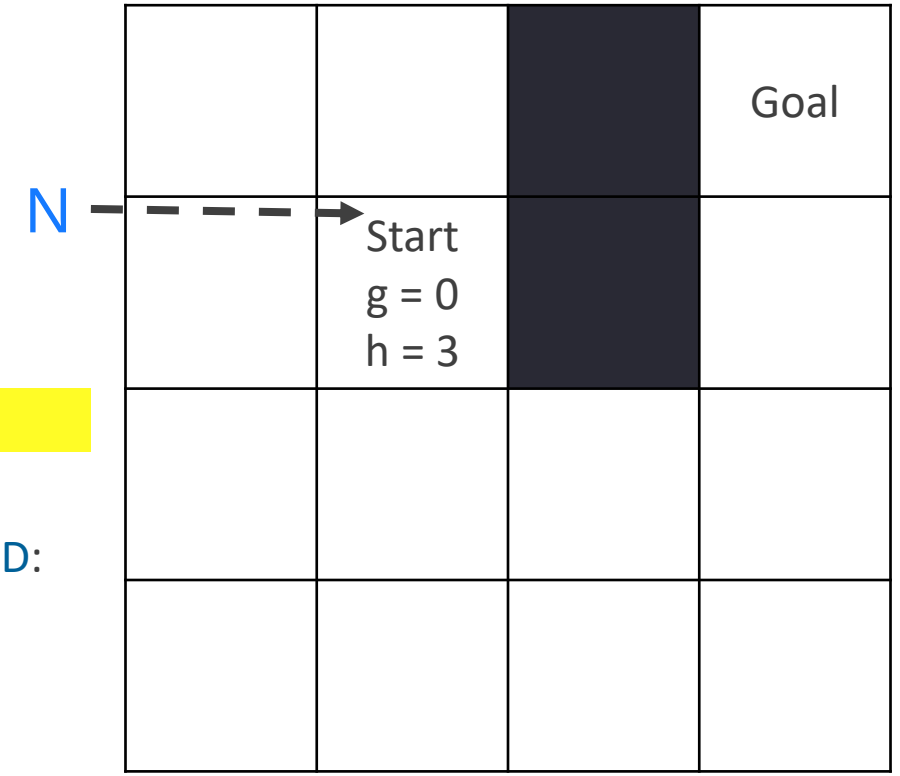


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

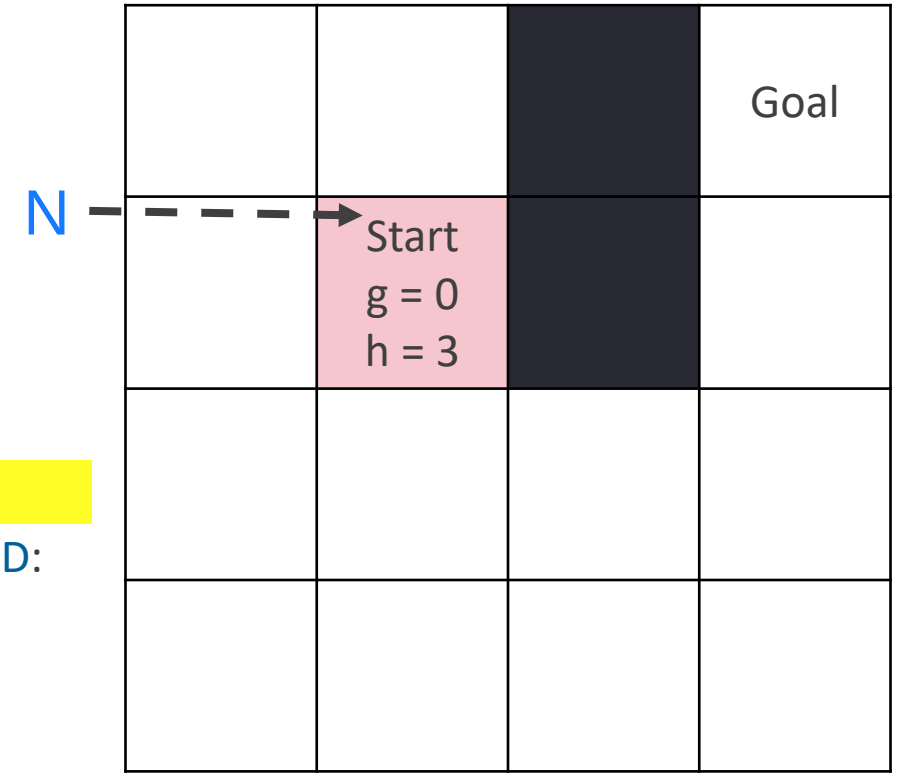
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```



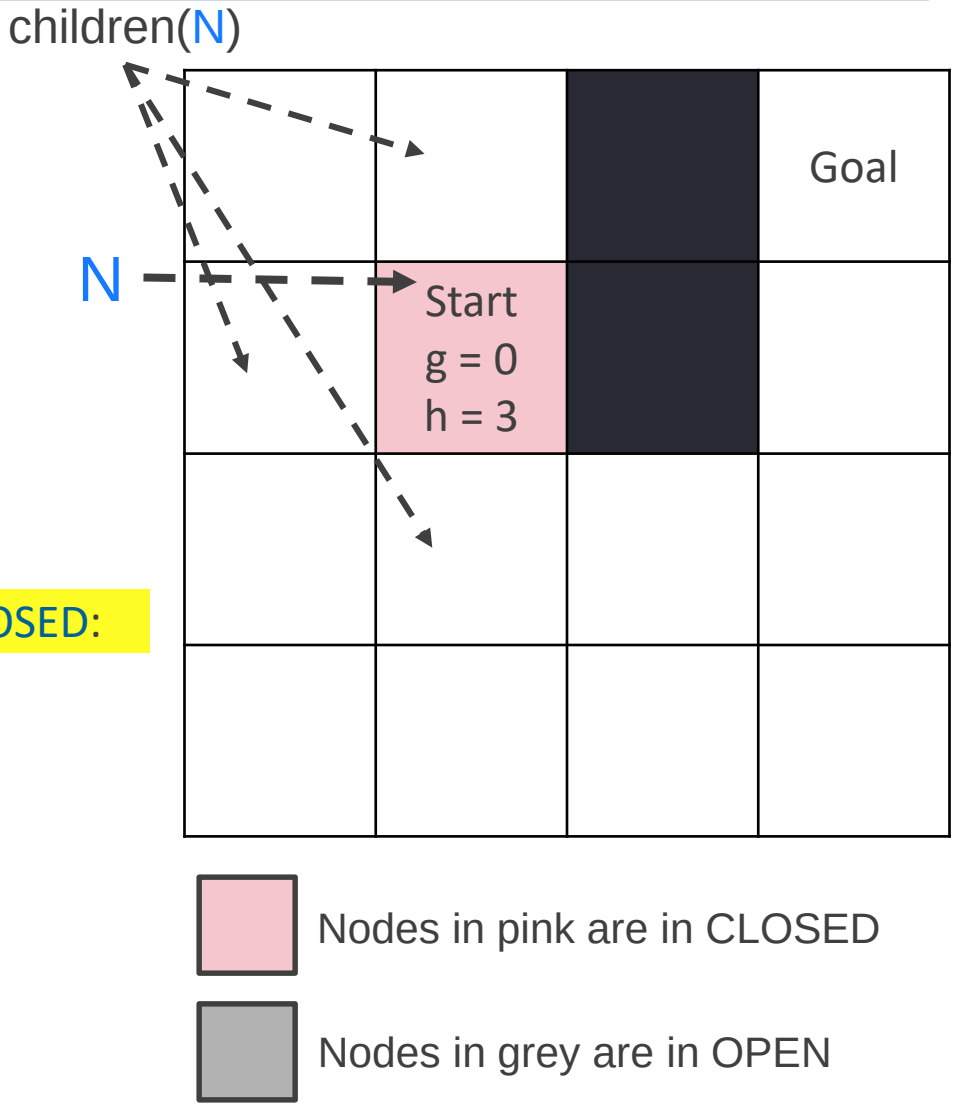
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
    CLOSED.add(N)
    FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
    ENDFOR
ENDWHILE
```



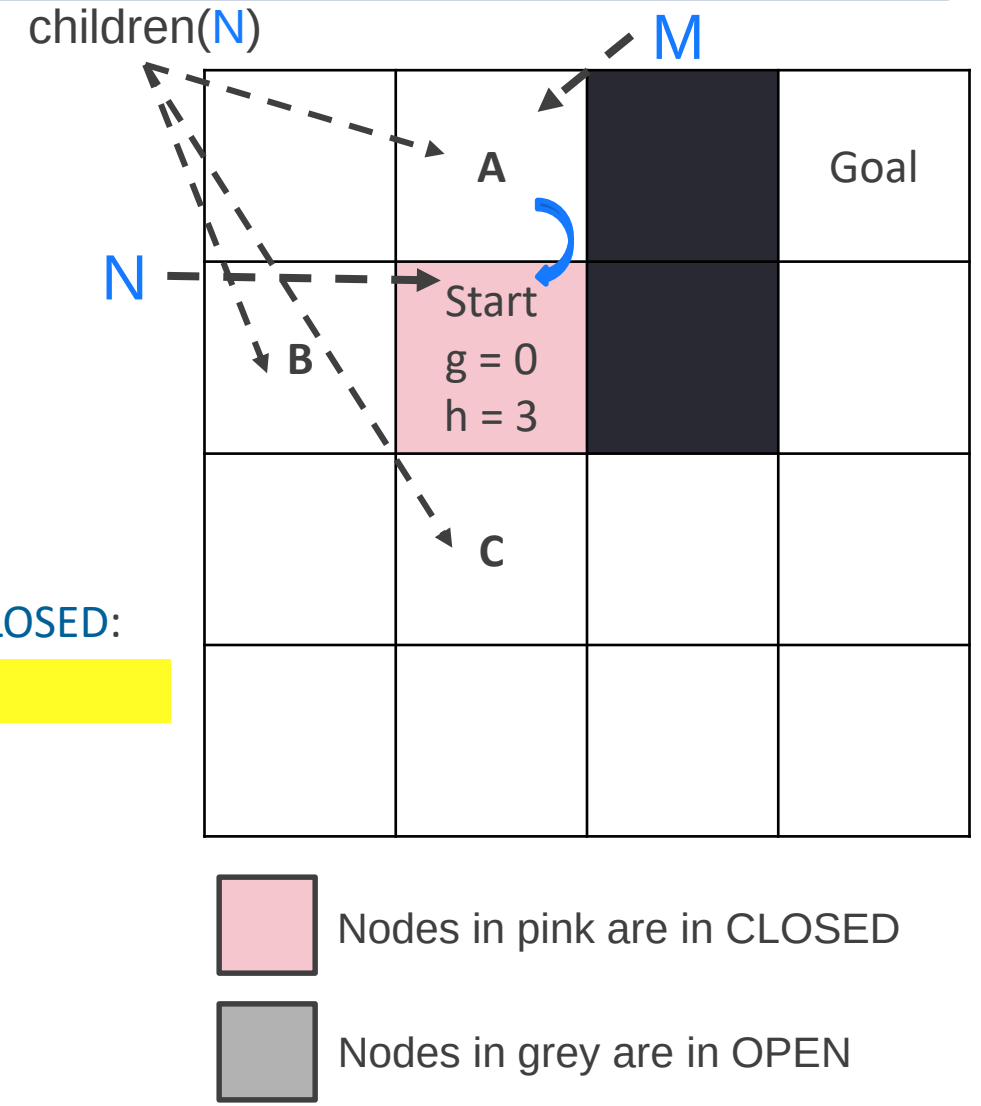
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```



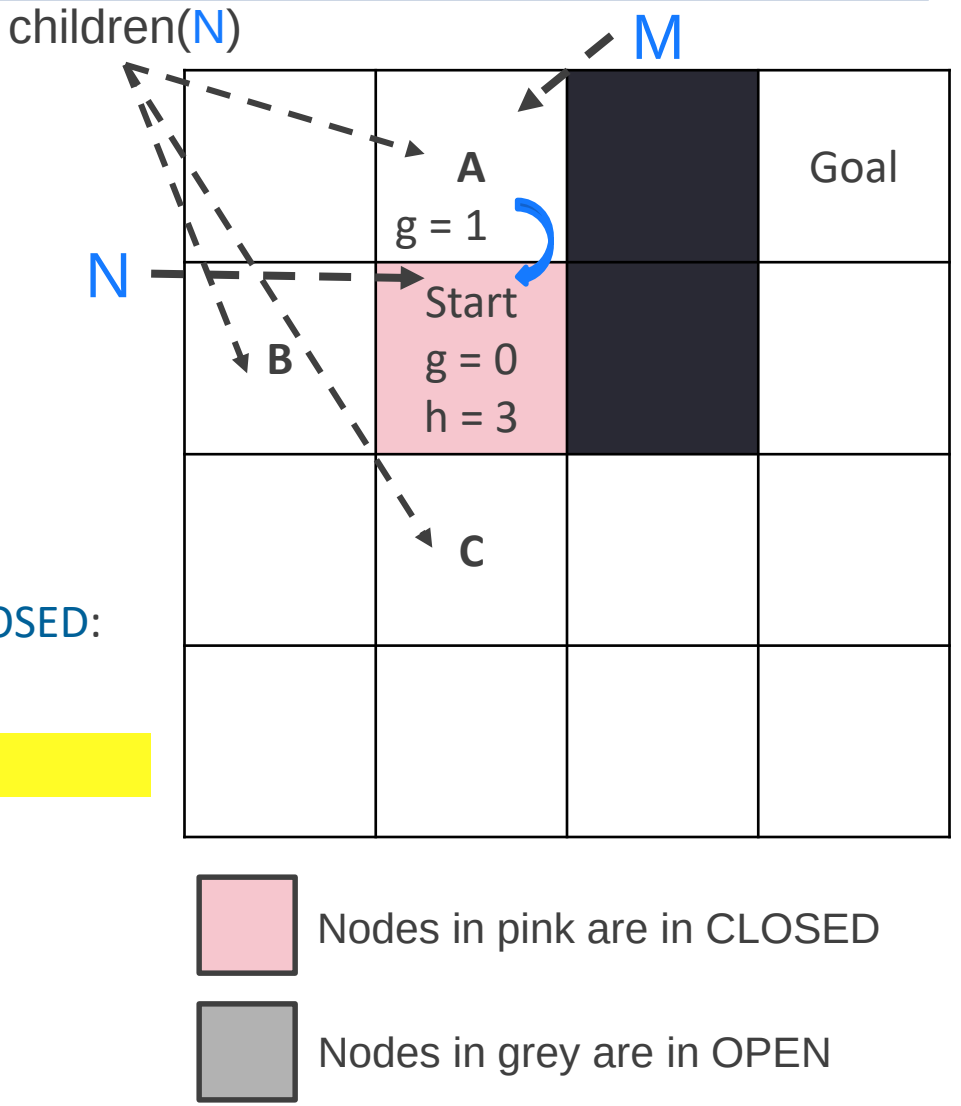
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```



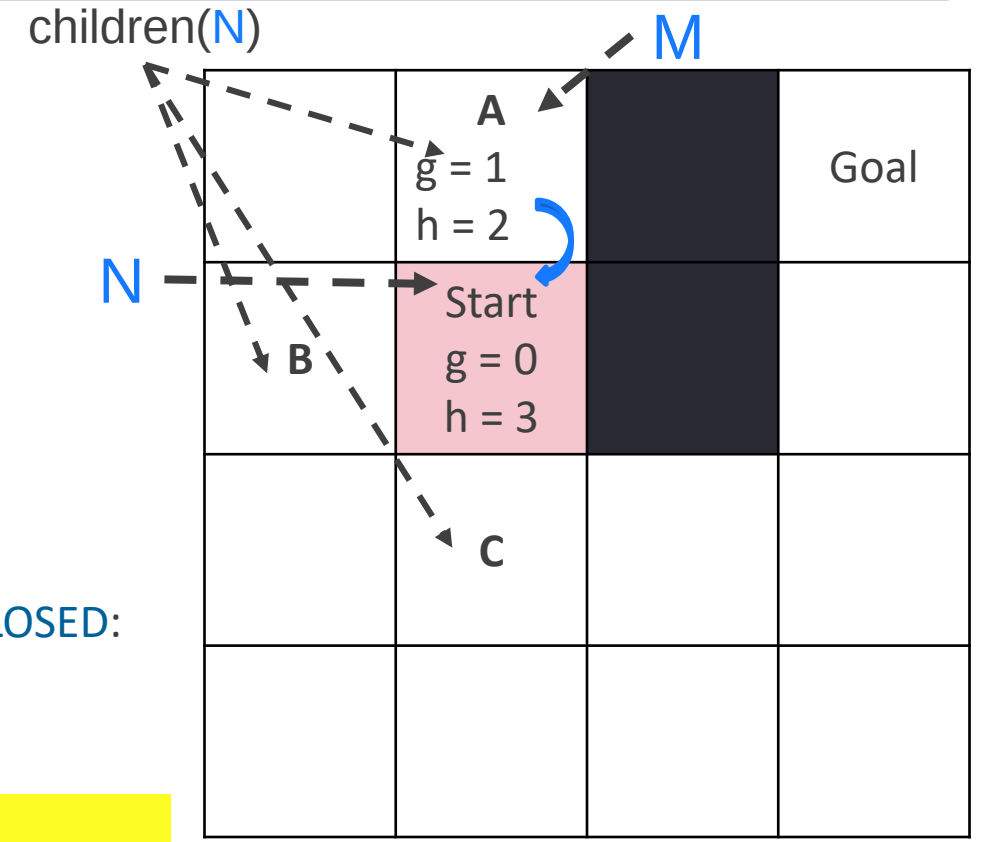
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```



A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

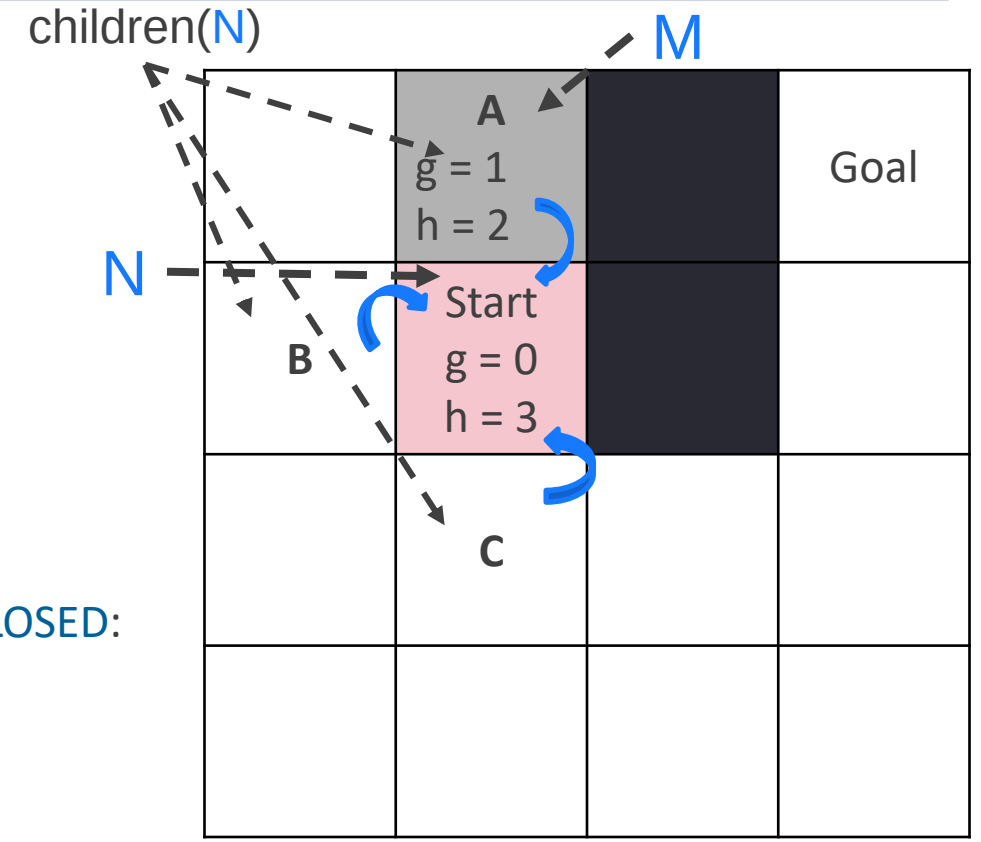


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

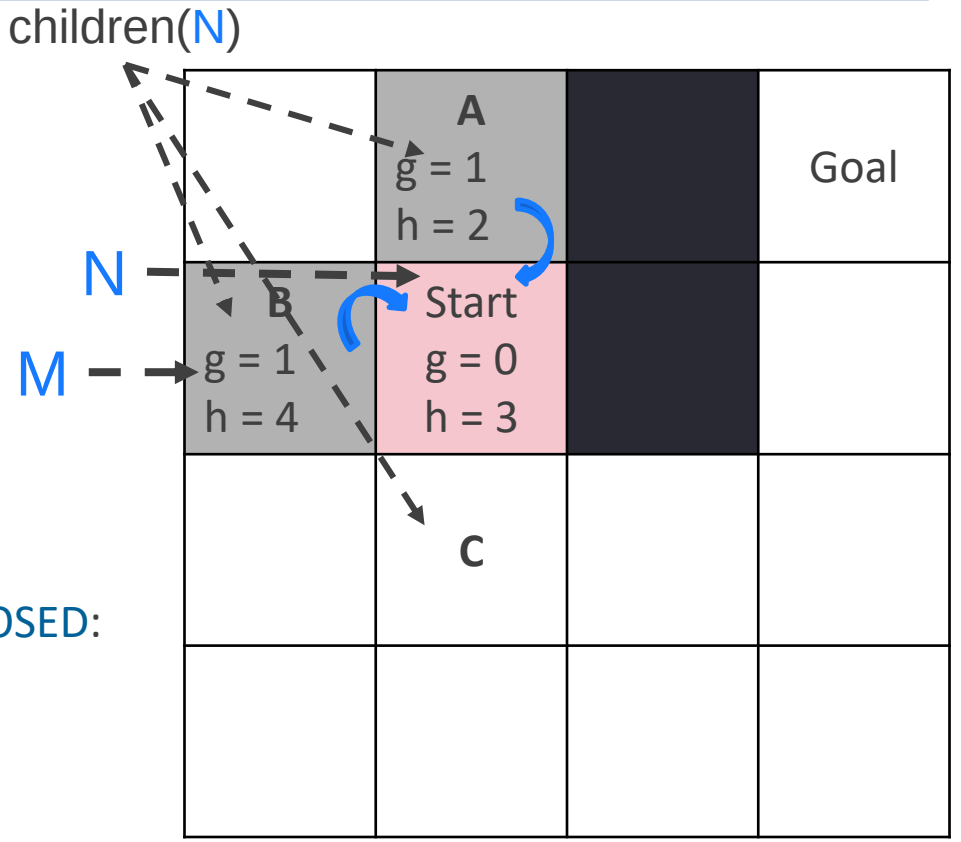


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

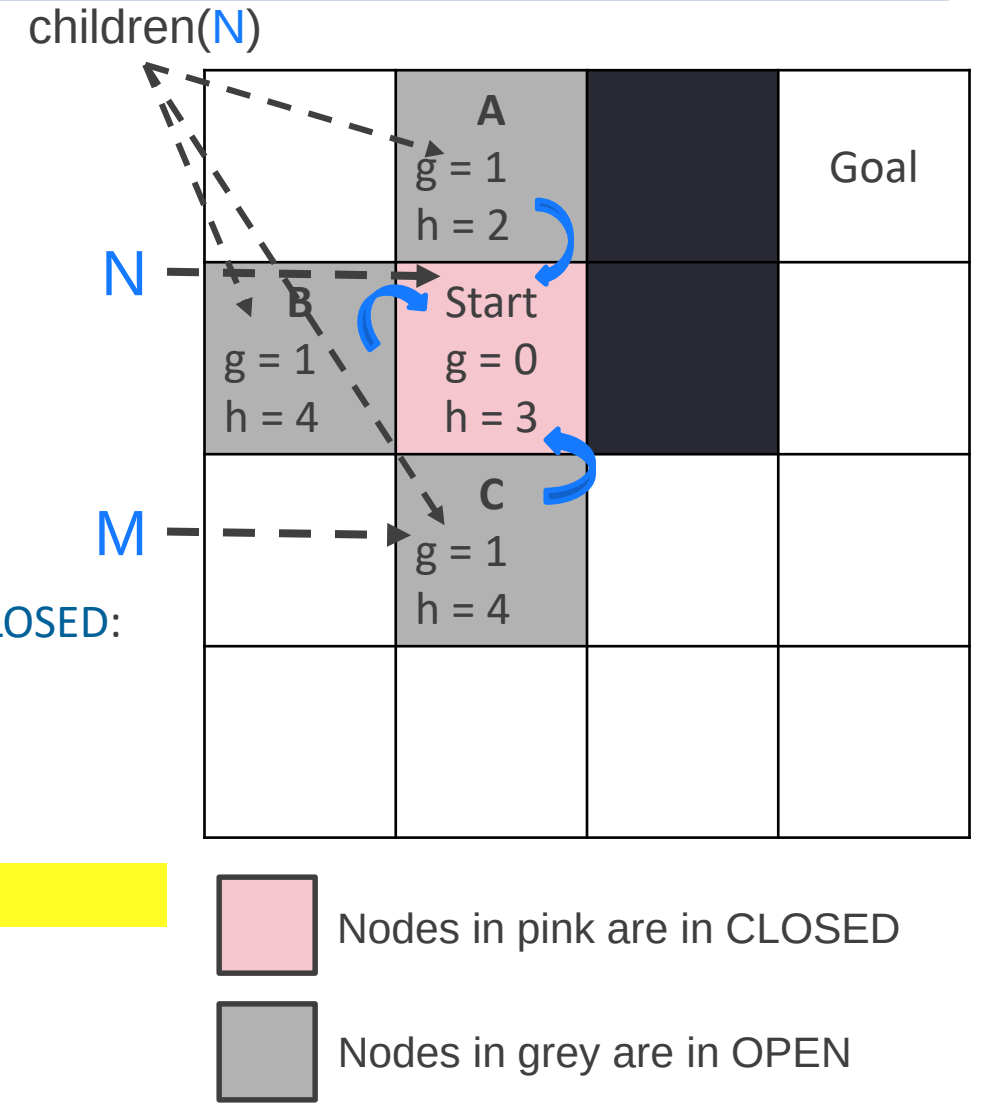


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

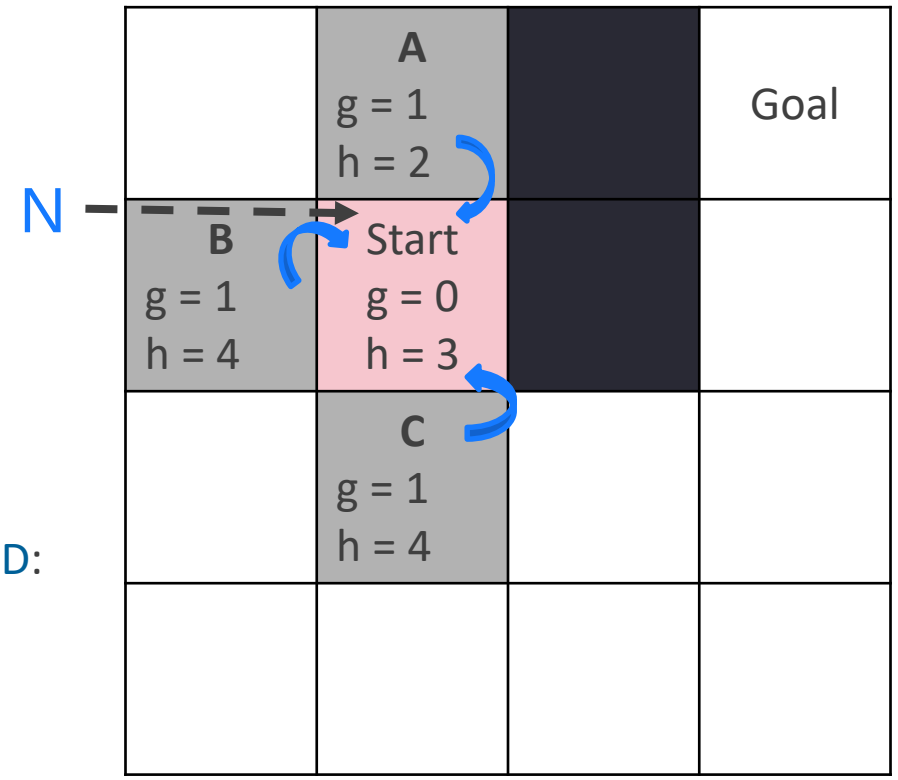
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```



A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

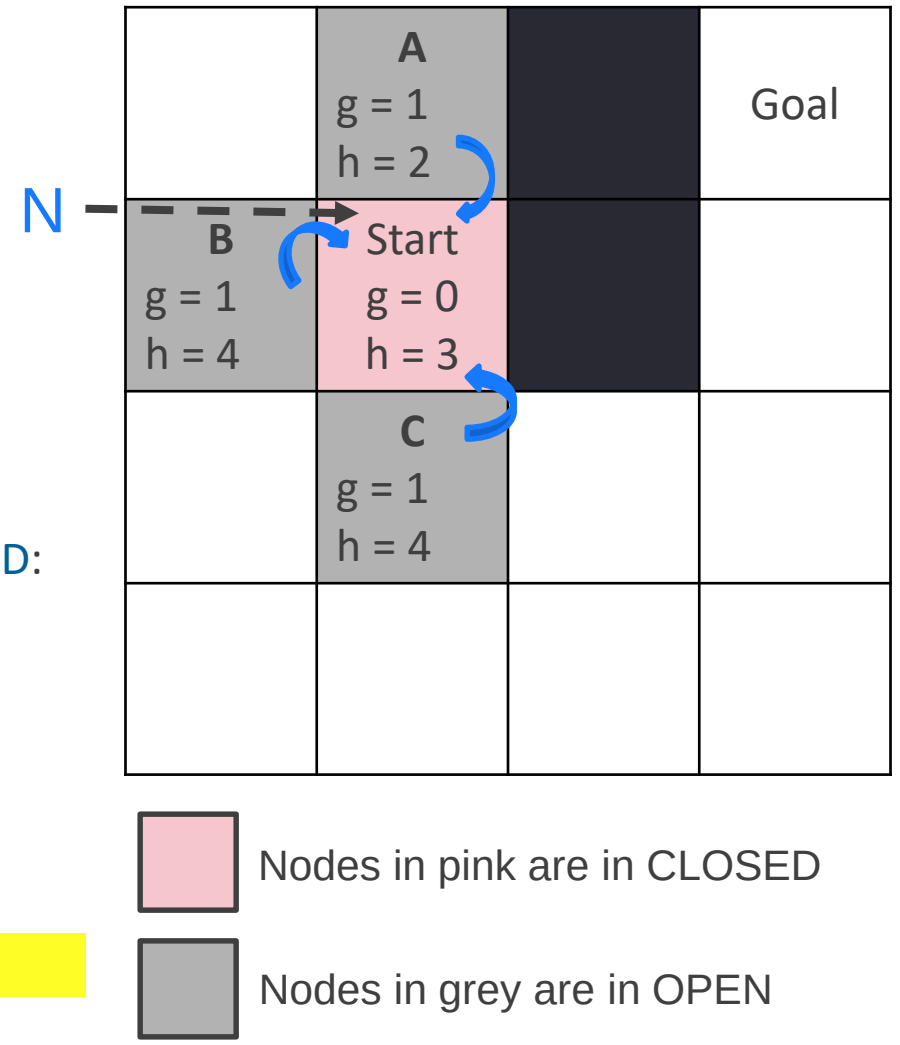


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

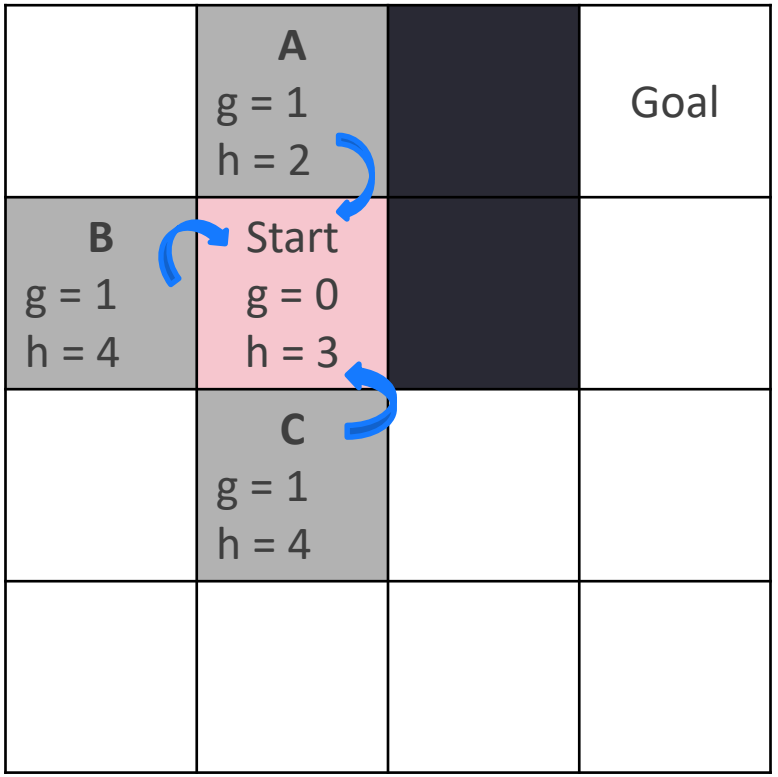
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```



A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

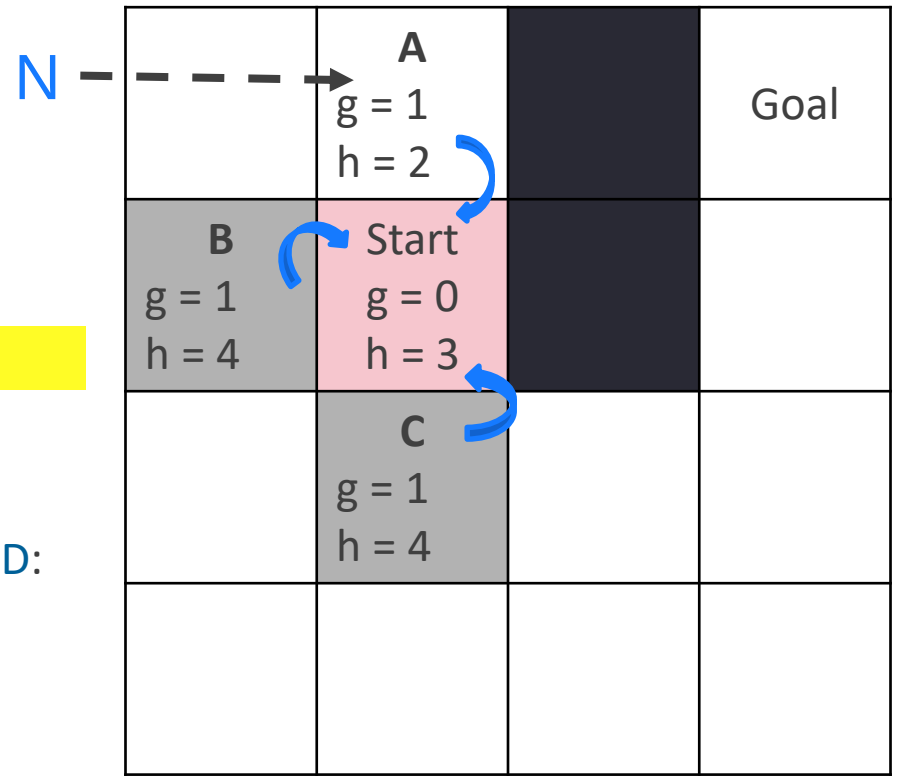


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

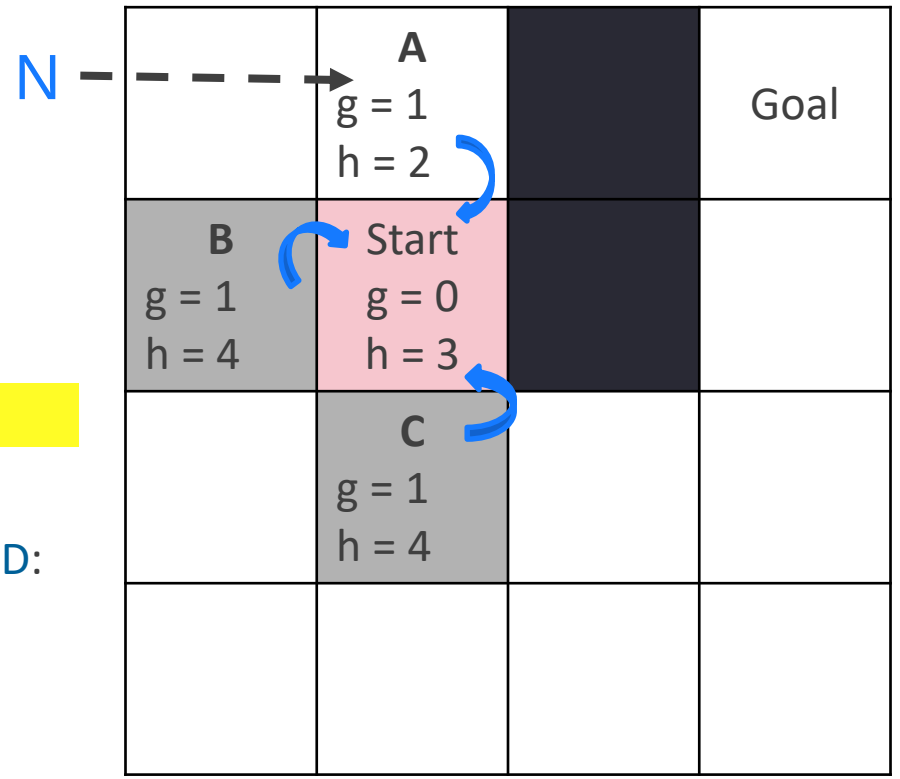


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

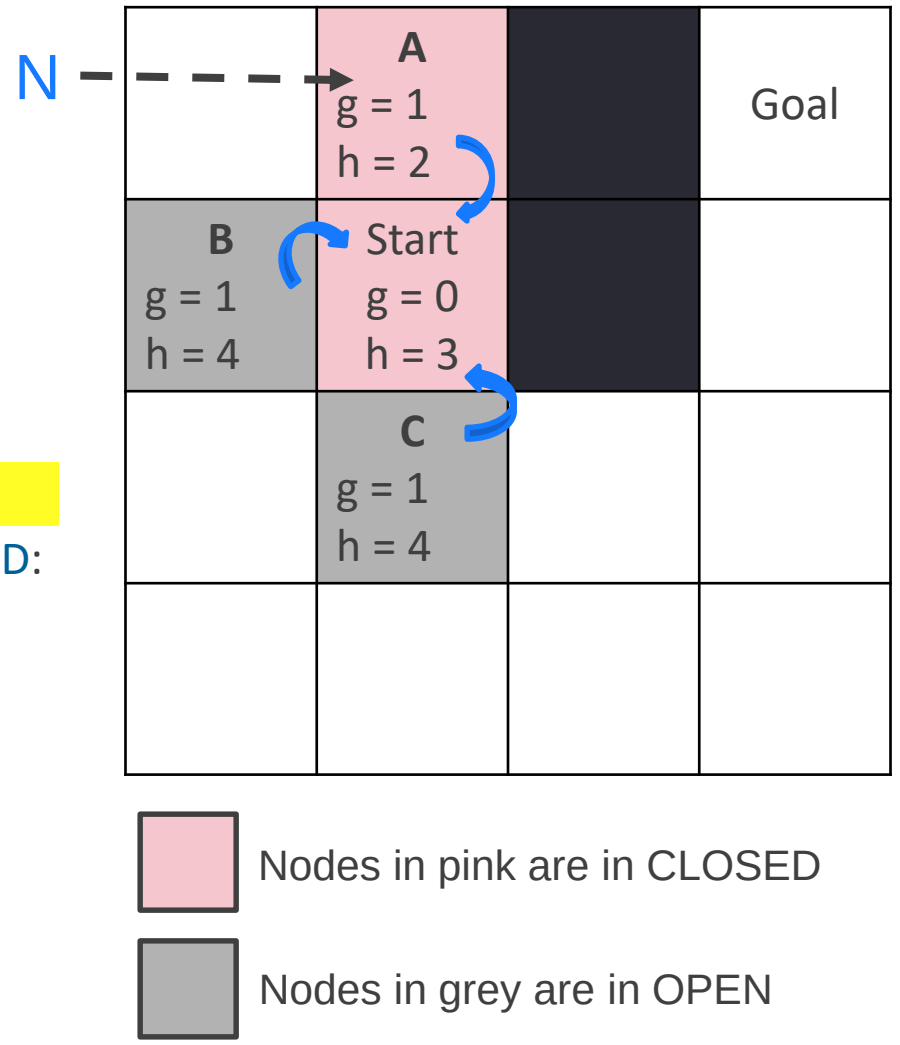


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
    CLOSED.add(N)
    FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
    ENDFOR
ENDWHILE
```



Properties of A*

- Complete – Yes, unless there infinitely many nodes with $f \leq f(G)$
- Time – Exponential in [relative error in h x length of solution]
- Space – Keeps all nodes in memory
- Optimal – Yes, cannot expand f_{i+1} until f_i is finished

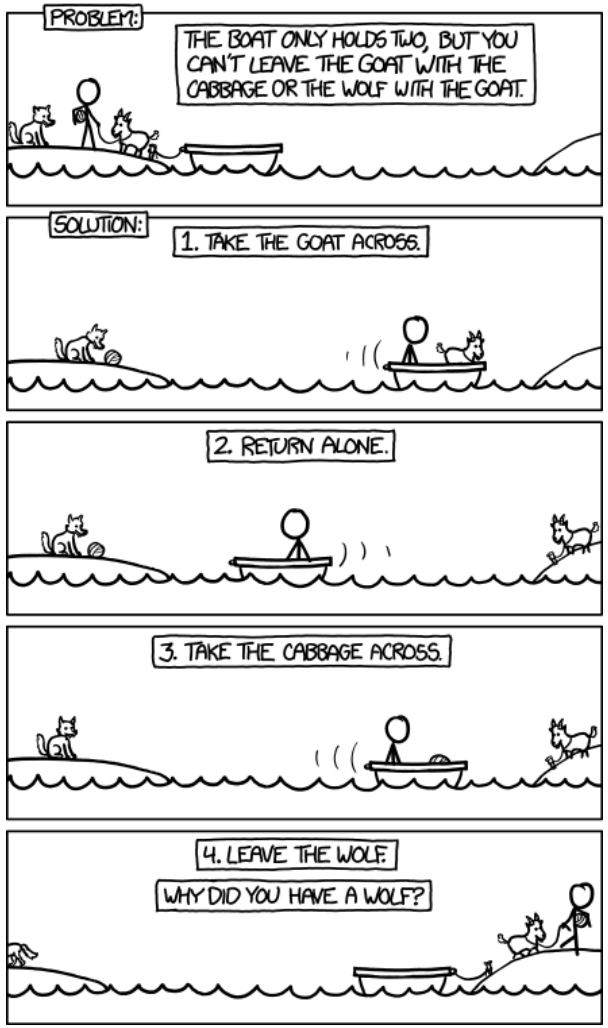
Admissible Heuristics

- To ensure optimality, A* requires the heuristic to be **admissible**:

$$h(n) \pounds h^*(n)$$ ← Actual cost to the goal

- In other words: the heuristic **must never overestimate** the actual remaining cost to the goal.

Heuristics: Wolf-Goat-Cabbage



- Let's assume that "cost" is the number of boat trips.
- What would be a good heuristic?
- Example:
  - Number of "items" on the left bank (since we know we need to make at least that number of trips)
  - Better:
    - ([*number of items in left bank*] – 1) * 2
    - Add 1 if boat on right bank and [*number of items in left bank*] > 0

Heuristics: 8-Puzzle

8		1
3	2	6
7	4	5

- Cost: number of moves
- What would be a good heuristic?
  - Heuristic 1:
    - Number of blocks out of place
  - Heuristic 2:
    - Distance of each block to its target position

Admissible Heuristics

- To ensure optimality, A* requires the heuristic to be **admissible**:

$$h(n) \pounds h^*(n)$$ ← Actual cost to the goal

- In other words: the heuristic **must never overestimate** the actual remaining cost to the goal.
- If the heuristic is admissible, A* will be ***provably*** optimal.
  - But an inadmissible heuristic might work just fine.

Heuristic Dominance

- If $h_2(n) \geq h_1(n)$ for all n (both admissible)

  Then h_2 dominates h_1 and is better for search
- Typical Search Costs:
  - d = 14 IDS = 3,473,941 nodes
    - $A^*(h_1)$ = 539 nodes
    - $A^*(h_2)$ = 113 nodes
  - d = 24 IDS ≈ 54,000,000,000 nodes
    - $A^*(h_1)$ = 39,135 nodes
    - $A^*(h_2)$ = 1,641 nodes
- Given any admissible heuristics h_a, h_b
  - $h(n) = \max(h_a(n), h_b(n))$ is also admissible and dominates h_a, h_b

Constructing Heuristics by Relaxation

$h_1(n)$ = number of tiles out of place

$h_2(n)$ = Manhattan distance of tiles to proper locations

- Admissible heuristics can be derived from the exact solution cost of a relaxed version of the problem
- If the rules of the 8-puzzle are relaxed so that a tile can move anywhere, the $h_1(n)$ gives the shortest solution
- If the rules are relaxed so that a tile can move to any adjacent square, then $h_2(n)$ gives the shortest solution

Key point: The optimal solution cost of a relaxed problem is not greater than the optimal solution cost of the real problem

A Different Way to Build Heuristics

- **Pattern Databases** – Store the exact optimal solution cost for every possible subproblem instance.
  - You care about some features of the problem, but not all
  - In this example, all the possible locations of [1,3,6,*blank*]
  - Identify the instance in the DB to get the h value

Start

	3	6
1	*	*
*	*	*

Goal

1	*	3
*	*	6
*	*	

Variations of A*

- SMA*: A* with bounded memory usage
- TBA*: A* for real-time domains where we have a bounded time before producing an action
- LRTA*: another real-time version of A* (very simple, and the basis of a whole family of algorithms)
- D*: A* for dynamic domains (problem configuration can change)
- etc.

Coming up…

- Next time, we'll look at some search variations and alternatives, including local search and constraint satisfaction